



Universitat
de les Illes Balears

TRABAJO DE FIN DE GRADO

ESTUDIS EXPLORATORIS LAIA@UIB: APLICACIÓ DE MODELS D'INTEL·LIGÈNCIA ARTIFICIAL PER A LA CLASSIFICACIÓ D'IMATGES DE CÀLCULS RENALS

Khaoula Ikkene

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Any acadèmic 2025-26

ESTUDIS EXPLORATORIS LAIA@UIB: APLICACIÓ DE MODELS D'INTEL · LIGÈNCIA ARTIFICIAL PER A LA CLASSIFICACIÓ D'IMATGES DE CÀLCULS RENALS

Khaoula Ikkene

Trabajo de Fin de Grado

Escola Politècnica Superior

Universitat de les Illes Balears

Any acadèmic 2025-26

Paraules clau del treball: Cálculos renales, LBP, extracción de parches, balanceo de datos

Tutor: Antoni Jaume Capó, Miquel Miró Nicolau

Supervisor: Bernat Isern Amengual

Quiero expresar mi más sincero agradecimiento a mis tutores, Toni y Miquel, así como a mi supervisor, Bernat, por su asesoramiento, su paciencia y el acompañamiento que me han brindado a lo largo de la realización de este trabajo. Agradezco también al Grupo de Investigación en Litiasis Renal de la Universitat de les Illes Balears por la labor desarrollada durante años, que ha hecho posible disponer del conjunto de muestras sobre el que se ha podido construir este TFG. Por último, quiero dar las gracias a mi familia: a mis padres y a mi hermana, por su apoyo constante y por motivarme siempre a seguir aprendiendo, y a mi abuela que, aunque ya no está con nosotros, siempre será mi fuente de inspiración y fortaleza.

ÍNDICE GENERAL

Índice general	III
Índice de figuras	V
Índice de tablas	VII
Acrónimos	IX
Resumen	XI
1 Introducción	1
1.1. Motivación del proyecto	5
1.1.1. Objetivo general	5
1.1.2. Objetivos específicos	5
1.2. Planificación del trabajo	6
1.3. Estructura de la memoria	7
1.4. Declaración del uso de la IA	7
2 Marco teórico	9
2.1. Marco clínico	9
2.1.1. Proceso de clasificación	9
2.2. Extracción de características	11
2.2.1. Espacios de color	11
2.2.2. Descriptores de textura basados en LBP	11
2.2.3. Descriptores de color basados en HSV	12
2.3. Extracción de parches	13
2.4. XGBoost	14
2.5. Redes Neuronales Convolucionales	15
2.6. MedSAM	19
3 Metodología	23
3.1. Entorno de ejecución	23
3.2. Conjunto de datos	24
3.2.1. Caracterización estadística del conjunto	25
3.3. Segmentación	27
3.4. Diseño experimental	28
3.4.1. Modelos seleccionados	28
3.4.2. Modelo híbrido: <i>HybridResNet50</i>	28

3.4.3.	Imágenes completas frente a parches	29
3.5.	Tareas de clasificación	29
3.6.	Extracción de parches	30
3.6.1.	Algoritmo de extracción de parches	30
3.6.2.	Selección del tamaño del parche	31
3.6.3.	Agregación de predicciones a nivel de imagen	31
3.7.	Gestión del desequilibrio de clases	31
3.7.1.	Caso de imágenes	32
3.7.2.	Caso de parches	32
3.8.	Métricas de evaluación	32
3.9.	Validación cruzada	34
3.9.1.	Agrupación de imágenes por cálculo	34
3.10.	Búsqueda de hiperparámetros	34
3.11.	Visualización del espacio de características	35
3.12.	<i>Pipelines</i>	36
3.12.1.	Pipeline del XGBoost	36
3.12.2.	Pipeline de la ResNet50	36
3.12.3.	Pipeline del HybridResNet50	36
3.12.4.	Evaluación de los modelos	36
3.13.	Reproducibilidad	37
4	Resultados y discusión	39
4.1.	Mejores parámetros (P, R) del LBP	39
4.2.	Tamaño idóneo del parche	40
4.3.	Clasificación usando imágenes	42
4.3.1.	Clasificación binaria	42
4.3.2.	Clasificación multiclase	43
4.4.	Clasificación usando parches	49
4.4.1.	Clasificación binaria	49
4.4.2.	Clasificación multiclase	50
4.4.3.	Imágenes completas frente a parches	52
4.5.	Comparación con el estado del arte	53
4.6.	Estudio de ablación: histogramas de color frente a energías HSV	55
5	Conclusiones	57
5.1.	Rendimiento de los modelos	57
5.2.	Contribuciones del trabajo	58
5.3.	Implicaciones clínicas	58
5.4.	Limitaciones	59
5.5.	Líneas de trabajo futuras	60
5.6.	Reflexión personal	60
A	Annexos	63
A.1.	Parámetros de los modelos del DL	63
	Bibliografía	65

ÍNDICE DE FIGURAS

2.1. Ejemplos de vecinos con simetría circular para diferentes pares (P, R). Extraído de [1].	12
2.2. Estructura del bloque residual. Extraído de [2].	18
2.3. A la izquierda, un bloque residual básico. A la derecha, un bloque de construcción del bloque cuello de botella para ResNet50/101/152. Extraído de [2].	19
3.1. Muestras de imágenes agrupadas por clase y vista.	25
3.2. Distribución por clase del área, la cobertura, el número de fragmentos y el tamaño del fragmento principal de los cálculos.	26
3.3. Segmentación con MedSAM: ejemplo de una imagen con su máscara generada y la correspondiente imagen de superposición.	27
3.4. Diagrama general del pipeline experimental que ilustra los modelos usados con los dos tipos de entrada y las técnicas de balanceo usadas en cada escenario.	29
3.5. Ejemplo de parches y sus máscaras extraídos a partir de una imagen con cobertura mínima del 90 % y con tamaño de 224 x 224.	31
4.1. Gráfico comparativo de las métricas ponderadas AUC, exactitud y <i>F1</i> para diferentes valores de (P, R) del XGBoost en el escenario de clasificación binaria de las imágenes.	40
4.2. Métricas ponderadas (exactitud, <i>F1</i> y AUC) obtenidas para los diferentes tamaños de parche evaluados con el modelo XGBoost en el escenario de clasificación binaria.	41
4.3. Métricas ponderadas (exactitud, <i>F1</i> y AUC) obtenidas para los diferentes tamaños de parche evaluados con el ResNet50 en el escenario de clasificación binaria.	41
4.4. Proyección UMAP 2D de la capa de clasificación de los tres modelos en el escenario de clasificación binaria de imágenes.	43
4.5. Proyección UMAP 2D de la capa de clasificación de los tres modelos en el escenario de clasificación multiclase de imágenes.	44
4.6. Matrices de confusión, agregadas sobre los <i>folds</i> , de la clasificación multiclase sobre imágenes completas.	45
4.7. Proyección del espacio de características LBP+HSV de las imágenes originales (puntos) y sus versiones aumentadas (cruces), coloreadas por clase.	48
4.8. Proyección UMAP 2D de la capa de clasificación de los tres modelos en el escenario de clasificación multiclase de parches.	52

ÍNDICE DE TABLAS

2.1. Clasificación morfo-constitucional de los cálculos renales propuesta por Grases <i>et al.</i> [3]. En caso de grupos con los tipos no diferenciados (4 y 9) por el nombre se usa el nombre de su subtipo. Adaptado de [3].	10
3.1. Distribución del conjunto de datos procesado por clase y vista.	25
3.2. Espacio de búsqueda de Optuna para los hiperparámetros de XGBoost. . .	35
4.1. Resultados comparativos del rendimiento de los modelos en clasificación binaria y multiclase (versión ponderada) bajo <i>StratifiedGroupKFold</i> con $K = 5$ folds. Los mejores resultados de cada tarea se destacan en negrita . .	42
4.2. Resultados por clase (Precisión, Exhaustividad y FI , en %) para los tres modelos bajo la misma estrategia de validación. Los valores se expresan como media $\pm$ desviación estándar, en porcentaje; la columna Soporte indica el número de muestras por clase.	45
4.3. Resultados de las técnicas de balanceo aplicadas a XGBoost para la clasificación multiclase de imágenes.	47
4.4. Resultados detallados por clase (Precisión, Exhaustividad y FI , en %) para el modelo XGBoost comparando la estrategia Baseline frente a Baseline+SMOTE. Los valores se expresan como media $\pm$ desviación estándar, en porcentaje. La columna Soporte indica el número de muestras por clase. .	47
4.5. Resultados de clasificación binaria (COM vs. Otros) sobre parches, a nivel de parche y agregados por imagen. Los mejores resultados de cada bloque se destacan en negrita	49
4.6. Comparación de las métricas de los modelos con y sin aumentación de parches en la clasificación multiclase a nivel de parche. Los mejores resultados de cada tarea se destacan en negrita	50
4.7. Resultados de la clasificación multiclase sobre parches, a nivel de parche y agregados por imagen. La columna a nivel de parche corresponde a la configuración sin aumentación de la tabla 4.6. Los mejores resultados de cada tarea se destacan en negrita	51
4.8. Resultados detallados por clase (Precisión, Exhaustividad y FI , en %) de los tres clasificadores evaluados en el escenario de clasificación multiclase de parches. Los valores se expresan como media $\pm$ desviación estándar, en porcentaje. La columna Soporte indica el número de muestras por clase. .	51
4.9. Comparación del FI entre la clasificación sobre imágenes completas y la clasificación sobre parches agregada por imagen, para ambas tareas. . . .	53

4.10. Correspondencia aproximada entre los tipos de la clasificación de Daudon y los de la clasificación de Grases.	54
4.11. Resultados por clase (Precisión, Exhaustividad y <i>F1</i> , en %) para los modelos evaluados en el trabajo de referencia de López <i>et al.</i> [4]. Los valores de <i>F1</i> se han calculado a partir de la precisión y la exhaustividad reportadas en su tabla de resultados por clase.	54
4.12. Resultados de validación cruzada (5 <i>folds</i>) comparando energías HSV e histogramas de color sobre imágenes completas, agrupados por tarea. Las métricas AUC, <i>F1</i> , precisión y exhaustividad corresponden a sus variantes ponderadas. Los valores se expresan como media $\pm$ desviación estándar, en porcentaje. Los mejores resultados de cada tarea se destacan en negrita	55
4.13. Resultados de validación cruzada (5 <i>folds</i>) comparando energías HSV e histogramas de color con el XGBoost sobre parches, a nivel de parche, agrupados por tarea. Las métricas AUC, <i>F1</i> , precisión y exhaustividad corresponden a sus variantes ponderadas. Los valores se expresan como media $\pm$ desviación estándar, en porcentaje. Los mejores resultados de cada tarea se destacan en negrita	55
A.1. Hiperparámetros utilizados para el entrenamiento del ResNet50 y el modelo híbrido.	63

ACRÓNIMOS

AUC Area Under the Curve

BICUIB Biobanc de Càlculs Renals de la Universitat de les Illes Balears

BRU Brushite

COD Calcium Oxalate Dihydrate

COM Calcium Oxalate Monohydrate

CNN Convolutional Neural Network

DL Deep Learning

GPU Graphics Processing Unit

HAP Hydroxyapatite

HSV Hue, Saturation, Value

IA Inteligencia Artificial

LBP Local Binary Pattern

MCA Morpho-Constitutional Analysis

SL Shallow Learning

SMOTE Synthetic Minority Over-sampling Technique

TFG Trabajo de Fin de Grado

UA Uric Acid

UMAP Uniform Manifold Approximation and Projection

ViT Vision Transformer

XGBoost eXtreme Gradient Boosting

RESUMEN

En el tratamiento de los cálculos renales, reconocer el tipo del cálculo es importante para prescribir un tratamiento adecuado y evitar la recurrencia de la litiasis renal. Esta clasificación se realiza mediante el análisis morfoconstitucional, una técnica tediosa que puede tardar semanas. Por ello, se han llevado a cabo varias investigaciones para automatizar este proceso mediante modelos de inteligencia artificial. El presente trabajo se suma a ellas para explorar la capacidad de estos modelos y valorar si pueden servir de apoyo en el diagnóstico y tratamiento de la litiasis renal. Asimismo, constituye el primer contacto con un conjunto de datos del Biobanco de Cálculos Renales de la UIB. El trabajo emplea 397 imágenes *ex-vivo*, clasificadas según la taxonomía de Grases. Se definen dos tareas, una binaria y otra multiclase, y se emplean además dos tipos de entrada: las imágenes completas del cálculo y los fragmentos extraídos de estas (parches). Sobre esta base se comparan tres clasificadores: un modelo de aprendizaje superficial con descriptores manuales de color y textura; una red neuronal profunda; y un modelo híbrido que fusiona ambas fuentes de información. Para abordar el desbalance de clases se aplican distintas técnicas, ninguna de las cuales mejora el rendimiento. Los resultados destacan la capacidad del modelo híbrido en el escenario multiclase, donde alcanza un F1 del 74 %. En la clasificación binaria, en cambio, los tres modelos exhiben rendimientos equivalentes, con valores de hasta el 86 % en F1 y del 93 % en AUC. Asimismo, se revela que la limitación principal no reside en el desbalance de clases, sino en su solapamiento, por lo que cabe esperar que el acceso a un conjunto más amplio y sin las limitaciones identificadas mejore el rendimiento en trabajos futuros.

INTRODUCCIÓN

La litiasis renal —también denominada nefrolitiasis— es una enfermedad que consiste en una alteración de las condiciones naturales de cristalización de la orina [5]. Es una patología multifactorial en la que intervienen factores diversos como factores relacionados con la composición de la orina u otros ligados a la morfoanatomía renal [5]. A escala global, la prevalencia varía según la región: entre el 7 % y el 13 % en Norteamérica, el 5 % y el 9 % en Europa, y el 1 % y el 5 % en Asia [6]. En España, la prevalencia se sitúa en torno al 14,6 % y la incidencia en el 2,9 % en la población de entre 40 y 65 años [7]. Asimismo, esta enfermedad presenta una elevada tasa de recurrencia: en torno al 35 % de los pacientes desarrolla un nuevo episodio, con cifras notablemente superiores en composiciones específicas como la brushita o los cálculos de infección [8]; en seguimientos a largo plazo, la recurrencia alcanza aproximadamente el 50 % [9].

El análisis de los componentes del cálculo es clave para comprender las causas de su formación y, por tanto, intervenir correctamente y prevenir su recurrencia [3, 10]. En este contexto, Daudon *et al.* [11] establecieron en 1993 la primera clasificación morfoconstitucional de los cálculos renales, basada en la observación de que su morfología y composición conservan las huellas de las condiciones fisicoquímicas urinarias responsables de su formación. La clasificación distingue siete tipos principales y veintidós subtipos, cada uno asociado a etiologías metabólicas específicas [11, 12]. A pesar de su sólido valor diagnóstico, la complejidad del sistema ha dificultado su adopción sistemática en la práctica clínica habitual. En respuesta a esta limitación, en 2002 el grupo liderado por F. Grases [3] propuso una clasificación simplificada, basada en la composición y la micromorfología del cálculo y adaptada a la práctica clínica para facilitar la toma de decisiones terapéuticas.

La determinación del tipo de cálculo requiere técnicas de laboratorio capaces de caracterizar tanto su composición química como su microestructura. Las más utilizadas son la espectroscopia infrarroja por transformada de Fourier (FTIR), la difracción de rayos X y la microscopía electrónica de barrido (SEM) [10]. Su combinación con el examen morfológico mediante estereomicroscopio se denomina análisis morfoconstitucional (MCA, por sus siglas en inglés), considerado actualmente el método

de referencia para el estudio etiológico de los cálculos urinarios [13]. A diferencia de los métodos puramente composicionales, que se limitan a identificar la naturaleza química del cálculo, el MCA aporta también información sobre las condiciones que han favorecido su formación, lo que resulta esencial para orientar el tratamiento. Sin embargo, presenta varias limitaciones importantes [14]. Por un lado, el procedimiento puede tardar semanas en proporcionar los resultados del análisis. Por otro lado, el proceso de extracción de los fragmentos resulta laborioso. Estas restricciones han impulsado el interés por automatizar el proceso de clasificación de los cálculos renales.

La clasificación automática de cálculos renales mediante visión por computador ha evolucionado a lo largo de la última década en varios ejes: del análisis *ex-vivo* en condiciones controladas al reconocimiento *in-vivo* durante ureteroscopias reales. Las imágenes *ex-vivo* se obtienen tras la extracción quirúrgica del cálculo, normalmente en laboratorio y bajo condiciones de iluminación, fondo y resolución estandarizadas, lo que facilita la captura de ambas vistas (superficie y sección). Las imágenes *in-vivo*, en cambio, se capturan directamente durante la intervención endoscópica, con peor calidad óptica, iluminación variable y presencia de fluidos o sangre; del aprendizaje superficial (SL, por sus siglas en inglés) con descriptores manuales al aprendizaje profundo (DL, por sus siglas en inglés) de extremo a extremo; y de los enfoques de vista única a las arquitecturas de fusión multi-vista. La revisión que sigue recorre esta progresión en orden cronológico.

El grupo de Serrat [15] presentó *myStone*, el primer sistema de clasificación automática de cálculos renales, desarrollado sobre un total de 454 imágenes distribuidas en 8 clases. Para entrenar un clasificador *Random Forest*, los autores emplearon descriptores basados en histogramas RGB e histogramas de patrones locales binarios (LBP, por sus siglas en inglés) invariantes a rotación, e incorporaron además información clínica adicional —el pH de la orina— para mejorar los resultados. El sistema alcanzó una exactitud ponderada global en torno al 63 % y demostró el potencial de combinar textura y color en la clasificación de imágenes de cálculos renales. No obstante, su dependencia de información clínica externa, que no siempre está disponible, limita su aplicabilidad en escenarios basados únicamente en imagen; a ello se añade que el sistema no incorpora estrategias específicas para gestionar el desbalance entre clases.

En [16] se propuso una red neuronal convolucional (CNN, por sus siglas en inglés) basada en ResNet-101 y entrenada con imágenes *ex-vivo* de 63 cálculos para clasificar cinco composiciones distintas: oxalato cálcico monohidrato (COM, por sus siglas en inglés), ácido úrico (UA, por sus siglas en inglés), estruvita, brushita (BRU, por sus siglas en inglés) y cistina. El modelo obtuvo una exhaustividad ponderada del 85 % y demostró que las CNN profundas son capaces de identificar la composición del cálculo a partir de fotografías digitales, si bien el reducido tamaño del estudio limita la representatividad de la distribución de clases y la robustez estadística de sus conclusiones.

El salto hacia datos *in-vivo* lo realizó Martínez *et al.* [17] con un estudio que abordó la clasificación de 3 tipos de cálculos, separando las imágenes de cada vista (superficie y sección) en subconjuntos independientes. Para incrementar el número de muestras, extrajeron parches de 200×200 píxeles, sobre los que aplicaron como extractor de características la combinación de histogramas LBP (textura) y energías HSV (color). Se entrenaron dos modelos de aprendizaje superficial —KNN y *Random Forest*—, siendo este último el de mejor rendimiento, con una exactitud del 89 % sobre las imágenes de

la vista de sección. Su trabajo aportó una primera demostración de la capacidad de este enfoque de características sobre imágenes adquiridas con endoscopia durante ureteroscopias reales, abriendo la vía para la clasificación automática de este tipo de imágenes. A pesar de ello, no se ha explorado la combinación de la información de ambas vistas para el entrenamiento de un modelo más robusto y cercano al proceso MCA.

López-Tiro *et al.* [14] constituye la referencia principal en este ámbito. En su primer trabajo establecieron una línea base relevante para los trabajos posteriores. Estudiaron el rendimiento de modelos de SL (*Random Forest* [18] y *XGBoost* [19]) y DL (AlexNet [20], VGG19 [21] e Inception [22]) para clasificar cuatro tipos de cálculos usando imágenes *in-vivo* de las cuales se extraen parches de tamaño 256×256 , evaluando tres modelos por cada vista: solo superficie, solo sección, y ambas. Los resultados fueron prometedores, con una precisión y una exhaustividad del 97 % y 98 % respectivamente alcanzados por Inception sobre el conjunto mixto de las imágenes, demostrando la viabilidad de la clasificación de cálculos incluso en entornos con datos escasos. Otra contribución relevante fue evidenciar la capacidad del *XGBoost* para realizar esta clasificación cuando se eligen características adecuadas de color y textura. Asimismo, este trabajo constituye el primer intento en la exploración de las imágenes de ambas vistas del cálculo para entrenar los clasificadores.

Estrade *et al.* [23] analizaron el rendimiento de las CNN (ResNet-152-V2 [24]) para clasificar imágenes endoscópicas intraoperatorias distribuidas en cinco clases que engloban tanto cálculos puros como mixtos, con 347 imágenes de superficie y 236 de sección. El modelo alcanzó exactitudes medias superiores al 87 %. Adicionalmente, los mapas Grad-CAM [25], una técnica que utiliza los gradientes de cualquier concepto objetivo que fluyen hacia la última capa convolucional para producir un mapa de localización aproximada que resalta las regiones de la imagen importantes para predecir dicho concepto [25], mostraron que en el 98 % de las clasificaciones correctas la red atendía a regiones del propio cálculo, lo que aporta interpretabilidad y validez de la capacidad de la CNN para discriminar morfologías puras y mixtas en un entorno clínico real. Este trabajo tiene dos limitaciones esenciales: no emplea ambas vistas del cálculo (superficie y sección) durante el entrenamiento, y no aplica técnicas de balanceo de clases más allá de la aumentación de datos sobre el conjunto de entrenamiento completo.

Los trabajos posteriores han explorado principalmente dos líneas de mejora. La primera es la fusión de vistas múltiples: Villalvazo-Avila *et al.* [26] propusieron incorporar mecanismos de atención al final de cada bloque convolucional para fusionar información de las vistas de superficie y sección, emulando el proceso del MCA. Esta estrategia mejoró los resultados de los *backbones* individuales en un 4 % de media, y la fusión de características profundas mejoró los resultados globales hasta en un 11 % respecto al estado del arte previo. Sin embargo, el desequilibrio entre clases (algunas con apenas 4 imágenes en una de las vistas) se aborda mediante extracción aleatoria de un mayor número de parches de las clases minoritarias hasta alcanzar los 1000 parches por clase, sin estrategias específicas adicionales. En la misma línea, López-Tiro *et al.* [27] extendieron posteriormente este enfoque incorporando un proceso de transferencia de aprendizaje en dos etapas y bloques de atención, logrando mejoras de más del 6 % en precisión respecto a los modelos de vista única. A pesar de estos resultados prometedores, los autores reconocen como limitación principal el reducido tamaño

del conjunto y el escaso número de clases.

La segunda línea aborda el problema de la escasez de datos etiquetados. Zhu *et al.* [28] propusieron *SegPrompt*, un método que utiliza mapas automáticos de segmentación con un doble propósito: facilitar al modelo el reconocimiento de las regiones de interés y servir como *prompts* para ajustar modelos preentrenados [28]. Este enfoque logra un mejor equilibrio entre capacidad de ajuste y generalización con datos de entrenamiento limitados, y reduce el número de parámetros entrenables respecto al *fine-tuning* convencional. En la misma dirección, en otro trabajo [29] se evaluaron métodos de *few-shot learning* [30], un tipo de aprendizaje que permite a los modelos generalizar su conocimiento a partir de un número muy reducido de datos, normalmente entre una y veinte muestras por clase. Estos métodos se evaluaron para abordar el problema de la escasez de datos de entrenamiento que afecta a los modelos de DL en este dominio. Los aportes de esos trabajos, por tanto, no se trasladan directamente al escenario que aborda este TFG, en el que el conjunto cuenta con un número de muestras relativamente suficiente para un entrenamiento supervisado estándar.

El trabajo de referencia principal de este TFG es el de López-Tiro *et al.* [4], que compara seis métodos de aprendizaje superficial (SVM, AdaBoost, Bagging, MLP, Random Forest y XGBoost) y tres arquitecturas de aprendizaje profundo (AlexNet [20], VGG16 [21] e Inception-V3 [31]) sobre imágenes *in-vivo* de los cuatro tipos más frecuentes de cálculos urinarios. El modelo con mejor rendimiento fue Inception-V3 (precisión, exhaustividad y *F1* ponderados de 97%, 98% y 98% respectivamente), aunque también demostraron que, eligiendo el espacio de color y las características de textura adecuados, un método de aprendizaje superficial puede aproximarse al rendimiento del mejor modelo de aprendizaje profundo: el clasificador XGBoost alcanzó un *F1* ponderado del 96%. En concreto, la combinación de energías de color HSV e histogramas LBP invariantes a rotación fue identificada como la que aportaba mayor separabilidad entre clases, siendo *Random Forest* y XGBoost los clasificadores clásicos con mayor precisión y exhaustividad.

Finalmente, la incorporación de arquitecturas más recientes ha permitido nuevos avances. Reyes-Amezcuá *et al.* [32] realizaron un estudio comparativo entre transformer de visión (ViT, por sus siglas en inglés) [33] y CNN, demostrando que el modelo basado en ViT y preentrenado en ImageNet-21k supera sistemáticamente a ResNet50: en el subconjunto más complejo visualmente (parches de sección), el ViT alcanzó una exactitud del 95,2% y un *F1* del 95,1%, frente al 64,5% y 59,3% de ResNet50. En conclusión, el estudio compara distintas arquitecturas profundas entre sí, pero no incluye métodos superficiales como referencia ni aborda estrategias para el manejo del desbalance entre clases.

A pesar del progreso descrito, varios retos persisten: la mayoría de los trabajos con alto rendimiento se basan en datos *ex-vivo* o en conjuntos de datos privados de acceso restringido; y el desequilibrio de clases es un problema recurrente, condicionado por la menor frecuencia de algunos tipos de cálculo. El presente TFG aborda estos retos comparando la viabilidad de modelos de aprendizaje superficial y profundo para la clasificación de cálculos renales, investigando estrategias para mitigar el desequilibrio de clases y la escasez de muestras, y proponiendo un enfoque híbrido que fusiona las características aprendidas por la red con descriptores manuales de textura y color.

1.1. Motivación del proyecto

Este proyecto se suma a los trabajos orientados a automatizar la clasificación de los cálculos renales como complemento al diagnóstico clínico, ante las limitaciones del MCA descritas anteriormente.

Este enfoque resulta viable hoy gracias a los avances de la última década en visión por computador y aprendizaje profundo, así como a los resultados prometedores de los trabajos del estado del arte. En este contexto, disponer del conjunto de imágenes proporcionado por el Grupo de Investigación en Litiasis Renal de la Universitat de les Illes Balears supone una oportunidad para probar estos métodos en un escenario independiente y sentar una base experimental para futuros trabajos dentro del grupo.

A continuación se detallan los objetivos planteados en este TFG.

1.1.1. Objetivo general

Analizar la viabilidad de distintos modelos de inteligencia artificial para la clasificación automática de imágenes de cálculos renales, comparando enfoques de aprendizaje superficial, profundo e híbrido en términos de rendimiento, robustez y adaptabilidad a conjuntos de datos de tamaño reducido y con desequilibrio de clases.

1.1.2. Objetivos específicos

1. Preparar y homogeneizar el conjunto de datos aplicando un preprocesamiento para garantizar la calidad y consistencia de los datos de entrada para todos los modelos, incluyendo la segmentación automática de los cálculos, un análisis exploratorio de la distribución de clases y de la variabilidad de las imágenes.
2. Implementar y evaluar modelos de clasificación basados en IA de dos enfoques: aprendizaje superficial mediante XGBoost y aprendizaje profundo mediante ResNet50 *con transfer learning*. Para ambos se estudiará el rendimiento bajo dos esquemas de clasificación (binaria y multiclase), el efecto de usar imágenes frente a parches como entrada de los modelos, el impacto del aumento de datos, y la efectividad de distintas estrategias de balanceo de clases para ambos modelos.
3. Explorar un enfoque híbrido que combina los descriptores manuales del *pipeline* de XGBoost con las características aprendidas por el *backbone* de ResNet50, evaluando si dicha combinación aporta mejoras respecto a los modelos individuales.
4. Validar el rendimiento de los modelos utilizando un enfoque de evaluación común y riguroso, basado en validación cruzada y un conjunto de métricas que viabiliza la comparación de los resultados bajo distintos escenarios de clasificación.
5. Analizar los resultados de forma crítica para detectar las limitaciones del trabajo y sugerir líneas de mejora o trabajo futuro.

1.2. Planificación del trabajo

El desarrollo del TFG se ha organizado en cinco fases sucesivas, con varias tareas ejecutadas en paralelo para optimizar el calendario. A continuación se describe el contenido de cada fase.

Investigación y planificación inicial

La primera fase ha consistido en la búsqueda bibliográfica de los trabajos de referencia en clasificación automática de cálculos. A partir de esta revisión se han definido los enfoques que se desarrollarán a lo largo del proyecto y la estructura general del *pipeline*: los tres modelos a comparar (XGBoost con características LBP+HSV, ResNet50 con *transfer learning* y un modelo híbrido), los criterios de evaluación y la estrategia de validación.

Preparación del conjunto de datos

Esta fase ha cubierto el procesamiento del conjunto de imágenes y ha incluido:

- **Análisis exploratorio del conjunto de datos** que ha permitido caracterizar la variabilidad de las imágenes, depurar las inconsistencias detectadas e identificar sus principales limitaciones.
- **Búsqueda de un modelo de segmentación.** Inicialmente, se planteó utilizar un modelo preentrenado sin necesidad de *fine-tuning*, pero no se han encontrado opciones públicas adecuadas para el dominio. Finalmente, se ha adoptado un enfoque semiautomático basado en MedSAM y Label-Studio.
- **Separación de las imágenes según la vista.**

Desarrollo de los modelos

En paralelo a las últimas etapas de la fase anterior, se ha iniciado la programación del módulo de extracción de características (LBP+HSV) y del módulo de *transfer learning* sobre ResNet50. Una vez disponible el conjunto segmentado, se han ejecutado los primeros experimentos sobre el enfoque basado en imágenes, tanto en clasificación binaria como multiclase.

Refinamiento experimental

En esta fase se ha finalizado la programación de los componentes faltantes: extracción de parches, implementación de técnicas de balanceo, y la programación del enfoque híbrido tras decidir la estrategia de fusión.

Posteriormente, se ha llevado a cabo el conjunto completo de experimentos: comparación entre los tres modelos sobre las dos modalidades de entrada y para las dos clasificaciones definidas, y análisis de los mecanismos de balanceo.

Análisis de resultados y redacción

La última fase ha cubierto el análisis comparativo de los resultados, la generación de los gráficos y tablas que ilustran el comportamiento de cada modelo y la redacción de la presente memoria.

1.3. Estructura de la memoria

Esta memoria se estructura en cuatro capítulos principales: marco teórico, metodología, resultados y discusión, y conclusiones.

El primer capítulo cubre los conceptos necesarios para facilitar la lectura del trabajo. Comienza con una contextualización clínica del problema, que incluye una interpretación breve de la clasificación propuesta por Grases *et al.* [3] y empleada en este trabajo, y continúa detallando los conceptos relacionados con la extracción de parches y con los modelos de clasificación y segmentación abordados.

El segundo capítulo presenta la metodología seguida y justifica las decisiones tomadas: comienza describiendo el entorno de ejecución, el conjunto de datos y las decisiones de preprocesamiento aplicadas sobre él, continúa con el diseño experimental, el marco común de evaluación y termina con la explicación de los *pipelines* de los modelos.

El tercer capítulo presenta los resultados obtenidos, junto con su interpretación y la discusión correspondiente.

Finalmente, el cuarto capítulo recapitula las principales conclusiones y contribuciones del trabajo, e identifica sus limitaciones junto con las líneas de trabajo futuro que se derivan de ellas.

1.4. Declaración del uso de la IA

En este TFG se han empleado dos asistentes de inteligencia artificial. Por un lado, NotebookLM, para realizar búsquedas en profundidad de referencias del estado del arte, técnicas de balanceo, estrategias de extracción de parches, enfoques de fusión en modelos híbridos y *frameworks* para la búsqueda de hiperparámetros. Por otro lado, Claude, para tareas de paráfrasis, mejoras de redacción y estilo, así como para la asistencia en la depuración de código y su documentación y en el desarrollo de algunas funciones de visualización de gráficos. En todos los casos, el autor ha revisado todas las aportaciones antes de incorporarlas al trabajo.

MARCO TEÓRICO

Este capítulo abarca, brevemente, las nociones necesarias para facilitar la lectura del resto del TFG. Concretamente, se explica el marco clínico, los aspectos relacionados con la extracción de características y parches, y las nociones de los modelos de clasificación y segmentación empleados. Estos métodos se han seleccionado a partir de la revisión bibliográfica presentada en el Capítulo 1, priorizando aquellos que reportan buen rendimiento en escenarios comparables al de este trabajo: conjuntos de imágenes de cálculos renales de tamaño reducido y con desequilibrio entre clases. La justificación detallada de cada elección se desarrolla en el capítulo 3.

2.1. Marco clínico

El conjunto de datos proporcionado está clasificado siguiendo la taxonomía propuesta por Grases *et al.* [3]. A diferencia de las clasificaciones puramente compositivas, esta se basa en la combinación de tres criterios: la composición química global, la estructura interna fina del cálculo y la presencia o ausencia de un punto de inserción en la papila renal [3]. Gracias a ello, esta clasificación posee una relevancia clínica directa, ya que cada categoría puede asociarse a un proceso fisiopatológico específico y, por lo tanto, a una estrategia terapéutica diferenciada [3].

2.1.1. Proceso de clasificación

El protocolo definido por Grases *et al.* [3] combina técnicas de microscopía y análisis fisicoquímico aplicadas en distintas fases. Según describen los autores, se observa primero la estructura externa del cálculo mediante estereomicroscopio óptico para identificar características superficiales, luego se fragmenta por su zona central para visualizar la estructura interna, sobre la que se realizan dos análisis complementarios: espectroscopia infrarroja (IR) para determinar la composición química global, y microscopía electrónica de barrido (SEM) con microanálisis para caracterizar la microestructura y los microcomponentes presentes [3]. La integración de la información

2. MARCO TEÓRICO

morfológica, composicional y microestructural permite asignar el cálculo a una de las once categorías principales del sistema de clasificación [3].

Independientemente de su composición química, los cálculos renales pueden agruparse en dos categorías según el sitio donde se han formado: los **cálculos papilares**, que se desarrollan adheridos al epitelio papilar renal y los **cálculos de cavidad**, que se forman libremente en una cavidad renal, en ausencia de adherencia papilar [3].

Grupo	Frecuencia	Tipos y composición
Cálculos papilares de oxalato cálcico monohidrato (COM)	12,9 %	1a: núcleo COM/OM (60,7 %) 1b: núcleo HAP/OM (39,2 %)
Cálculos de COM no adheridos (formados en cavidades renales)	16,4 %	2a: núcleo de OM (63,3 %) 2b: núcleo de HAP (29,9 %) 2c: núcleo de ácido úrico (6,7 %)
Cálculos de oxalato cálcico dihidrato (COD) no adheridos	33,8 %	3a: COD con pequeñas cantidades de HAP (55,2 %) 3b: COD con poca OM (44,8 %)
Cálculos mixtos COD/HAP no adheridos	11,2 %	4I: capas alternas COD/HAP (39,6 %) 4II: depósitos desordenados COD/HAP (60,4 %)
Cálculos de hidroxiapatita (HAP) no adheridos	7,1 %	5a: con cantidades mínimas de COD (54,9 %) 5b: solo HAP y OM (45,1 %)
Cálculos infecciosos de estruvita	4,1 %	6: contienen también cantidades importantes de HAP y OM
Cálculos de brushita no adheridos	0,6 %	7: contienen también pequeñas cantidades de HAP
Cálculos de ácido úrico no adheridos	8,2 %	8a: ácido úrico anhidro (40,7 %) 8b: ácido úrico dihidratado (49,0 %) 8c: mixtos ácido úrico/uratos (8,8 %)
Cálculos mixtos oxalato cálcico/ácido úrico	2,6 %	9I: papilares (12,7 %) 9II: no adheridos / no papilares (87,3 %)
Cálculos de cistina no adheridos	1,1 %	10: contienen también pequeñas cantidades de OM
Cálculos infrecuentes	1,9 %	11a: OM como componente principal (32,6 %) 11b: de origen farmacológico (6,1 %) 11c: residuos post-LEOC (10,2 %) 11d: carbonato cálcico (14,3 %) 11e: artefactos (36,7 %)

Abreviaturas: OM: materia orgánica; LEOC: litotricia extracorpórea por ondas de choque.

Tabla 2.1: Clasificación morfo-constitucional de los cálculos renales propuesta por Grases *et al.* [3]. En caso de grupos con los tipos no diferenciados (4 y 9) por el nombre se usa el nombre de su subtipo. Adaptado de [3].

2.2. Extracción de características

2.2.1. Espacios de color

Un espacio de color es un modelo matemático utilizado para cuantificar y representar numéricamente la percepción humana del color.

Formalmente, un espacio de color (también denominado modelo de color o sistema de color) consiste en la especificación de un sistema de coordenadas y un subespacio dentro de dicho sistema, de modo que la representación de un color corresponde a la ubicación de un punto en ese subespacio [34].

RGB

El modelo RGB es un espacio de color aditivo diseñado para la captura, almacenamiento y visualización de imágenes digitales [34]. Geométricamente, se representa como un cubo unitario tridimensional donde los ejes corresponden a las luces primarias roja, verde y azul [35], y los colores se definen mediante una tupla de tres valores (R, G, B) . Aunque es el formato nativo de adquisición, este espacio es altamente sensible a los cambios de iluminación y carece de uniformidad perceptual, por lo que suele transformarse a otros espacios (HSV o L^*a^*b) para facilitar las tareas de visión por computadora [35].

HSV

Otra familia ampliamente utilizada en visión por computador es la HSV que descompone el color en tres componentes perceptualmente más intuitivos: el tono o matiz H (hue), la saturación S (saturation) y el valor V (value). El tono se representa como un ángulo en el rango $[0^\circ, 360^\circ]$, mientras que la matiz y el valor están en el rango $[0, 1]$. El primer elemento del HSV codifica el color puro con independencia de su intensidad o pureza, el segundo describe el grado de pureza de un color y el tercero representa la cantidad de luz o brillo del color [36].

La principal ventaja de HSV sobre RGB es la separación entre la información cromática y de luminosidad que facilita el diseño de descriptores robustos frente a variaciones de iluminación [4].

2.2.2. Descriptores de textura basados en LBP

LBP es un descriptor de textura ampliamente utilizado en el ámbito de la visión por computador para la clasificación de imágenes. Introducido por Ojala *et al.* [37], codifica la textura local mediante la comparación de los píxeles de una vecindad definida por P puntos de muestreo sobre una circunferencia de radio R alrededor de un píxel central (figura 2.1).

El proceso, descrito por Ojala [1], consiste en comparar el valor del píxel central con cada uno de sus vecinos: si el vecino tiene un valor mayor o igual que el del píxel central se le asigna un 1, en caso contrario recibe el valor 0. Estos valores binarios se concatenan de forma secuencial en un orden determinado formando un número binario que se asocia al píxel central. Este procedimiento se aplica a todos los píxeles de la imagen o región de interés, y los códigos resultantes se acumulan en un histograma

que constituye el vector de características. Los histogramas tienen contenedores que representan sus clases o categorías. En este contexto, cada contenedor corresponde a un código LBP distinto, y su valor es la frecuencia con que ese patrón aparece en la imagen o la región.

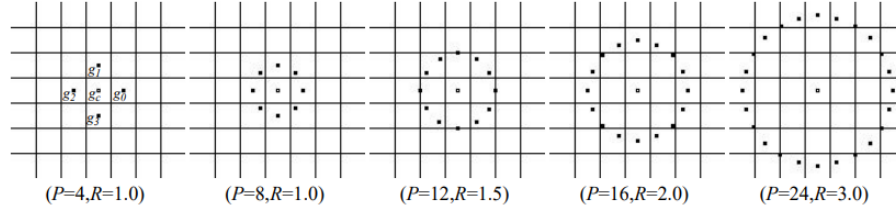


Figura 2.1: Ejemplos de vecinos con simetría circular para diferentes pares (P, R) . Extraído de [1].

El LBP es invariante en escala de grises [1], puesto que en su formulación el valor central de un píxel se calcula usando el orden relativo entre él y sus vecinos; así, cualquier transformación monótona de brillo o contraste no afecta ese orden local. Esta característica es de gran importancia, pues ofrece robustez a la iluminación, simplicidad computacional y discriminación potente en problemas reales de texturas con variaciones de brillo y contraste.

Sobre la formulación básica se han propuesto distintas extensiones. La primera de ellas es la noción de patrón uniforme, inspirada en la observación empírica de que la mayor parte de las ocurrencias de texturas naturales se concentran en un subconjunto reducido de patrones binarios [1]. Un patrón LBP, según se describe en el mismo trabajo, es uniforme si su código binario, recorrido de forma circular, contiene como máximo dos transiciones bit a bit (de 0 a 1 o de 1 a 0). El uso de estos patrones permite reducir la dimensionalidad del vector de características: se asigna un contenedor a cada patrón uniforme único y el resto de patrones no uniformes se agrupan en un único contenedor, ya que individualmente aportan poca información discriminativa [1].

En ese mismo trabajo, la invariancia a la rotación se obtiene considerando equivalentes a todas las rotaciones cíclicas de un mismo patrón binario. Al combinar esta extensión con la de los patrones uniformes se obtiene el operador denotado como LBP_{PR}^{riu2} , donde *riu* es abreviación de *rotation invariant uniform* y P es el número de puntos de vecindad de radio R .

Finalmente, existe una versión multiresolución del descriptor en la que se combinan varios pares de parámetros (P, R) para describir patrones a distintas escalas, cosa que permite captar texturas tanto finas como más gruesas dentro de un mismo vector de características [1].

2.2.3. Descriptores de color basados en HSV

Tras la conversión del RGB a HSV, se procede a calcular las energías de color por píxel y canal de la siguiente manera [14]:

Primero se calcula el gradiente ∇_x y ∇_y por cada píxel, donde $c \in [H, S, V]$.

$$\nabla_x(I_c(x, y)) = I_c(x + 1, y) - I_c(x - 1, y), \quad (2.1)$$

$$\nabla_y(I_c(x, y)) = I_c(x, y + 1) - I_c(x, y - 1), \quad (2.2)$$

Luego se obtiene la energía del píxel como

$$e_{I_c}(x, y) = \sqrt{\nabla_x(I_c(x, y))^2 + \nabla_y(I_c(x, y))^2}, \quad (2.3)$$

2.3. Extracción de parches

Un parche, en nuestro contexto, es una pequeña región extraída de una imagen de mayor tamaño. Los parches tienen forma rectangular o, más habitualmente, cuadrada y sirven para captar localmente determinados patrones, texturas o características.

La extracción de parches se aplica en distintos ámbitos. Una de sus aplicaciones más clásicas se encuentra en patología digital, donde la enorme resolución de las imágenes *whole-slide* hace inviable procesarlas como una única entrada y obliga a descomponerlas en parches más pequeños [38]. En otros dominios, como el presente en este trabajo, los parches se utilizan principalmente para aumentar el número efectivo de muestras de entrenamiento [4, 14, 27].

Existen varios enfoques para la extracción de parches, de los cuales se citan los siguientes:

Extracción basada en rejilla fija

En este enfoque, la imagen se recorre de modo sistemático mediante una cuadrícula de parches definida por dos parámetros principales: el tamaño del parche y el desplazamiento, entendido como la separación vertical u horizontal entre el origen de un parche y el del siguiente. El desplazamiento determina implícitamente el grado de solapamiento entre dos parches contiguos: cuando ese parámetro es igual al tamaño del parche, los parches son disjuntos. En caso contrario, los parches se solapan en una cantidad igual a la diferencia entre ambos valores.

Extracción aleatoria

En este enfoque se extraen los parches situando cada uno en una posición elegida aleatoriamente dentro de la imagen, pudiendo seleccionar el tamaño de parche y fijar, de antemano, la cantidad de parches a extraer de una imagen. A pesar de esta flexibilidad, la aleatoriedad en la que se basa esta estrategia no garantiza una cobertura uniforme del contenido de la imagen ni asegura que los parches contengan información relevante, especialmente si el objeto de interés ocupa solo una fracción de la imagen.

Extracción basada en regiones de interés (ROI)

En este último caso, los parches se extraen a partir de una zona previamente definida como relevante para la tarea, ya sea anotada manualmente o producida automáticamente por un algoritmo de segmentación. Dentro de esa región, la extracción puede realizarse mediante distintas estrategias: en algunas aplicaciones se determinan los centroides de los objetos de interés y se crea un parche centrado en cada uno de ellos [39]; en otras, se aplica un muestreo aleatorio dentro de la ROI con un solapamiento mínimo entre parches [40].

2.4. XGBoost

eXtreme Gradient Boosting (XGBoost) [19] es una implementación de *Gradient Boosting* ampliamente utilizada en problemas de aprendizaje supervisado. Para entender su funcionamiento se introducen, brevemente, los bloques sobre los que se construye: los árboles de decisión y el aprendizaje por conjuntos.

Un árbol de decisión es un modelo jerárquico que clasifica una muestra recorriendo una secuencia de particiones binarias hasta llegar a un nodo hoja, que indica la predicción final [41, 42]. El gran inconveniente de estos modelos es la alta varianza que presentan, ya que son sensibles a las pequeñas variaciones que puedan existir en los datos [41]. Por ello, se motiva el uso de los métodos por conjuntos, que combinan múltiples modelos para reducir la varianza y mejorar la generalización.

Dentro de los modelos por conjuntos, una forma posible de combinar los modelos usados es a través del **boosting** [19] que consiste en añadir nuevos modelos al conjunto de forma secuencial. En cada iteración, se entrena un nuevo modelo base débil teniendo en cuenta el error del conjunto completo aprendido hasta ese momento [43].

Formalmente, dado un conjunto de datos D de n muestras y m características $D = \{(x_i, y_i)\}$ con $|D| = n$, $x_i \in \mathbb{R}^m$ el vector de características de entrada de la muestra i , y $y_i \in \mathbb{R}$ la variable objetivo asociada a dicha muestra, un modelo de *boosting* se expresa de la manera [19]:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}. \quad (2.4)$$

donde $\hat{y}_i$ es el valor predicho por el modelo para la muestra x_i , $f_k(x_i)$ es la predicción aportada por el árbol k -ésimo al evaluarse sobre la entrada x_i , K es el número total de árboles, y $\mathcal{F}$ es el espacio de los árboles de regresión. La invariante más extendida del *boosting* es el *Gradient Boosting* [44] que consiste en aproximar el gradiente negativo de la función de pérdida con respecto a las predicciones acumuladas de los modelos por conjuntos en cada árbol, lo que equivale a un descenso de gradiente en el espacio de funciones [43].

XGBoost parte de este marco y se distingue del *Gradient Boosting* clásico por varias mejoras [19]. Concretamente, su función objetivo combina una función de pérdida diferenciable ℓ con un término explícito de regularización aplicado sobre cada árbol:

$$\mathcal{L}(\phi) = \sum_{i=1}^n \ell(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k), \quad (2.5)$$

La función de pérdida representada por $\ell(\hat{y}_i, y_i)$, mide la diferencia entre la predicción del modelo $\hat{y}_i$ y el valor real y_i para cada muestra i . Mientras que el término $\Omega(f_k)$ penaliza la complejidad estructural del árbol f_k , y se define como:

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|w\|^2, \quad (2.6)$$

siendo T el número de hojas del árbol f_k y w el vector de pesos asociado a dichas hojas. La función incluye dos hiperparámetros de regularización: γ que penaliza el número de hojas del árbol, controlando su complejidad estructural y λ que aplica una regularización sobre los pesos de las hojas para evitar que estos reciban valores extremos. De este modo, Ω actúa directamente sobre la estructura del árbol y favorece

la construcción de árboles más simples y menos expuestos al problema del sobreajuste [19].

Para encontrar la estructura óptima del árbol y los pesos de sus hojas, XGBoost aproxima la función de pérdida ℓ alrededor de la predicción acumulada del paso anterior $\hat{y}_i^{(t-1)}$, lo que permite la resolución del problema de optimización de forma analítica, obteniendo directamente el peso óptimo de cada hoja j del árbol:

$$w_j^* = - \frac{\sum_{i \in I_j} \frac{\partial \ell(\hat{y}_i^{(t-1)}, y_i)}{\partial \hat{y}_i^{(t-1)}}}{\sum_{i \in I_j} \frac{\partial^2 \ell(\hat{y}_i^{(t-1)}, y_i)}{\partial (\hat{y}_i^{(t-1)})^2} + \lambda}, \quad (2.7)$$

donde I_j es el conjunto de muestras que caen en la hoja j , el numerador es la suma de las derivadas primeras de la función de pérdida ℓ respecto a la predicción acumulada del paso anterior $\hat{y}_i^{(t-1)}$, y el denominador es la suma de las derivadas segundas de dicha función respecto al mismo término, sumada al coeficiente de regularización λ .

Por último, XGBoost sustituye la búsqueda exhaustiva de la mejor partición en un nodo, que evalúa todos los posibles puntos de corte de cada característica, por un algoritmo aproximado basado en cuantiles ponderados. Este enfoque agrupa los valores de la característica en un número reducido de intervalos (cuantiles), ponderados según la importancia de cada muestra en el cálculo del gradiente, y solo evalúa como candidatos los límites de dichos intervalos. Gracias a esta optimización, XGBoost puede escalar a grandes volúmenes de datos, reduciendo drásticamente el número de particiones a comparar en cada nodo sin pérdida de precisión [19].

2.5. Redes Neuronales Convolucionales

Las CNN son una familia de redes neuronales diseñadas para procesar datos con estructura espacial, como imágenes [45, 46]. Estas redes, basadas en una operación clave llamada convolución, se apoyan en tres ideas importantes [45]: la interacción dispersa, según la cual cada neurona se conecta únicamente con un subconjunto reducido de neuronas, cosa que reduce el coste de memoria y mejora la eficiencia; la compartición de pesos, que establece que los mismos parámetros se aplican repetidamente sobre distintas posiciones de la entrada; y la equivarianza a la traslación, que permite detectar un mismo patrón en cualquier región de la imagen.

Capa convolucional

La capa convolucional es el bloque central en la construcción de una CNN. Se encarga de la extracción de características aplicando operaciones lineales como las convoluciones, y otras no lineales como las funciones de activación [46].

Una convolución es una operación matemática que para una entrada dada llamada tensor, aplica una matriz pequeña llamada filtro sobre cada posición del tensor realizando una combinación lineal entre los valores de esos filtros y los de la región sobre la cual se sitúa, para producir una salida que se define como un mapa de características [46].

Formalmente, dada una imagen I y un filtro K de tamaño $k \times k$, la salida S en la posición (i, j) se calcula como:

$$S_{i,j} = (I * K)_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} K_{m,n} \cdot I_{s \cdot i - m + p, s \cdot j - n + p}, \quad (2.8)$$

donde i, j recorren las posiciones de la salida, m, n recorren las posiciones del filtro K , s es el paso con el que se desplaza el filtro sobre la imagen, y p es el relleno o *padding* añadido en los bordes de la imagen de entrada. Sobre la salida se aplica habitualmente una función de activación no lineal, típicamente ReLU.

Cada capa emplea F filtros distintos, generando un volumen de salida cuya profundidad coincide con dicho número de filtros. El tamaño espacial de la salida depende de los cuatro hiperparámetros mencionados (k, F, s y p) según la expresión:

$$O = \left\lfloor \frac{N - k + 2p}{s} \right\rfloor + 1, \quad (2.9)$$

donde N es el tamaño de la imagen de entrada y O el tamaño correspondiente de la salida.

Las CNN aprenden representaciones organizadas jerárquicamente: las capas iniciales detectan patrones de bajo nivel como ahora bordes, las intermedias usan esos patrones para buscar esquinas y contornos extensos, los cuales se reconocen como conjuntos de bordes. Finalmente, esa información permite la identificación completa del objeto en las capas finales [45].

Capa de *pooling*

Una capa de *pooling* aplica una operación de submuestreo que reduce el espacio dimensional de los mapas de activación, aporta cierta invariancia a pequeñas traslaciones y disminuye el número de parámetros aprendibles en las capas posteriores [46].

La operación se aplica de forma independiente sobre cada canal del volumen de entrada y divide el mapa en regiones disjuntas, típicamente de 2×2 , que se reducen a un único valor. Las dos variantes habituales son el **max pooling**, que conserva el valor máximo de la región y constituye la opción más extendida en las CNN modernas, y el **average pooling**, que calcula el promedio [46].

Las fórmulas de esas capas, en orden, se exponen a continuación:

$$\text{MaxPooling}(X)_{i,j,k} = \max_{m,n} X_{i \cdot s_x + m, j \cdot s_y + n, k}, \quad (2.10)$$

$$\text{AvgPooling}(X)_{i,j,k} = \frac{1}{f_x \cdot f_y} \sum_{m,n} X_{i \cdot s_x + m, j \cdot s_y + n, k}, \quad (2.11)$$

donde X es la entrada a la capa de *pooling*, con i, j las posiciones espaciales de la salida y k el índice del canal o profundidad. Los índices m, n recorren la ventana de *pooling*, de tamaño $f_x \times f_y$, que se desplaza sobre X con un paso de s_x y s_y en cada dimensión. En el caso del *MaxPooling*, la salida toma el valor máximo dentro de dicha ventana, mientras que en el *AvgPooling* la salida es el promedio de los valores contenidos en ella, normalizado por el número total de elementos de la ventana ($f_x \cdot f_y$).

Función de activación

Para evitar que la composición de múltiples capas de la red equivaldría a una única transformación lineal se usan las funciones de activación que mapean la salida de una neurona a un único valor e introducen no linealidad que concede a las redes neuronales su capacidad de modelar relaciones complejas entre entradas y salidas [45].

La función más utilizada actualmente es la ReLU (*Rectified Linear Unit*), empleada habitualmente en las capas intermedias u ocultas de la red, y se define como

$$f(z) = \max(0, z), \quad (2.12)$$

Permite convertir los valores negativos a 0 y conservar los positivos sin alteración [47].

Softmax es otra función no lineal ampliamente utilizada y se aplica en la capa de salida para problemas de clasificación multiclase. Definida como

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad (2.13)$$

donde K es el número de clases y z es la entrada de la capa de salida. Esta función transforma una entrada en una distribución, donde cada componente de la salida adquiere un valor dentro del intervalo $[0, 1]$ interpretado como una probabilidad, ya que la suma de todos los componentes de salida es igual a 1.

Batch normalization

El *batch normalization* [48] es una técnica de normalización que resuelve el problema del *internal covariate shift*: durante el entrenamiento, el cambio continuo de los pesos en cada capa hace que la distribución de las activaciones que reciben las capas posteriores varíe en cada paso, dificultando la convergencia del modelo.

La solución consiste en normalizar las salidas de cada capa convolucional antes de aplicar la función de activación. Para cada *mini-batch*, se calculan la media μ y la varianza σ^2 de las activaciones, y se normaliza cada activación z_i :

$$\hat{z}_i = \frac{z_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad (2.14)$$

donde ϵ es una constante pequeña que evita la división por cero. Posteriormente, se escalan los valores normalizados mediante dos parámetros aprendibles α y β , específicos de cada capa, para producir la salida o_i :

$$o_i = \alpha \hat{z}_i + \beta, \quad (2.15)$$

Estos parámetros permiten a la red recuperar, si lo necesita, una representación de mayor escala o desplazamiento que la estrictamente normalizada. En conjunto, el *batch normalization* permite entrenar redes más profundas con tasas de aprendizaje mayores, reduce la sensibilidad a la inicialización de los pesos y aporta un efecto regularizador [48].

ResNet

La arquitectura **ResNet** (*Residual Network*), introducida por He *et al.* [2], responde a una limitación observada al apilar más capas en redes profundas: una vez resuelto el desvanecimiento del gradiente mediante técnicas como la inicialización normalizada y el *batch normalization*, aumentar la profundidad de la red más allá de cierto punto incrementaba el error de entrenamiento sin que esto fuera provocado por el sobreajuste.

La solución propuesta por He *et al.* [2] consiste en reformular aquello que las capas aprenden, planteando lo denominado aprendizaje residual profundo: en lugar de exigir que un bloque de capas aprenda directamente una función objetivo $h(\mathbf{x})$ que transforme su entrada $\mathbf{x}$ en la representación deseada, se propone que el bloque aprenda únicamente la diferencia entre dicha función y la propia entrada.

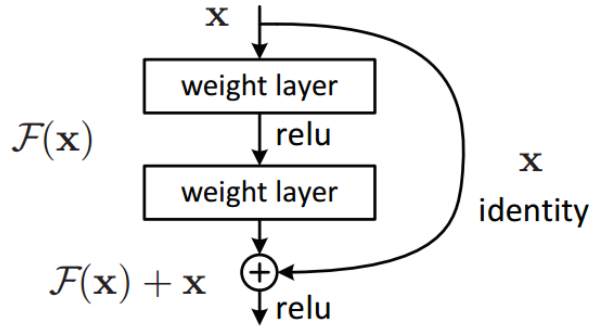


Figura 2.2: Estructura del bloque residual. Extraído de [2].

Para implementar este principio, se introduce el bloque residual [2] (véase la figura 2.2), formado por dos rutas paralelas —una rama principal que aplica una transformación no lineal $F(\mathbf{x})$ sobre la entrada, y una conexión de atajo que transporta directamente $\mathbf{x}$ sin modificarla— cuyas salidas se combinan mediante una suma elemento a elemento, y una activación ReLU se aplica tras esta suma para producir la salida del bloque:

$$o = f(x, \{W_i\}) + x, \quad (2.16)$$

donde o es la salida del bloque residual, y $\{W_i\}$ representa el conjunto de parámetros aprendibles, con i indexando cada una de dichas capas.

La presencia del término aditivo x facilita el flujo del gradiente hacia atrás durante el entrenamiento, al garantizar que una ruta de propagación que no atraviesa las capas no lineales de f y que, por lo tanto, no se ve afectada por la atenuación multiplicativa típica del *backpropagation* [2, 49].

Para reducir el coste computacional de redes muy profundas, las variantes de mayor profundidad de la familia (ResNet-50, ResNet-101 y ResNet-152) emplean una versión optimizada del bloque residual denominada bloque cuello de botella [2] (figura 2.3). Para cada función residual f se utiliza una estructura de tres capas: una convolución 1×1 que reduce el número de canales, una convolución 3×3 que aplica el procesamiento espacial sobre el volumen reducido, y una convolución 1×1 que expande de nuevo el número de canales hasta el original. Con este cambio, el procesamiento espacial costoso se aplica sobre la convolución 3×3 con un número reducido de canales, lo

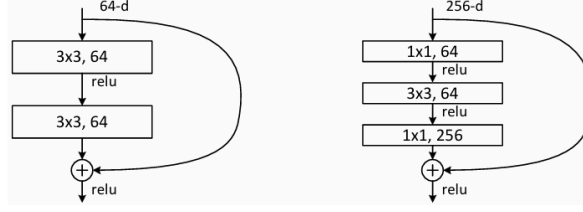


Figura 2.3: A la izquierda, un bloque residual básico. A la derecha, un bloque de construcción del bloque cuello de botella para ResNet50/101/152. Extraído de [2].

que disminuye drásticamente el número de parámetros sin sacrificar capacidad de representación [2].

2.6. MedSAM

Para introducir MedSAM [50], el modelo de segmentación empleado en el *pipeline* de este trabajo, es necesario cubrir previamente cuatro conceptos: codificador, decodificador, ViT y atención cruzada.

En el contexto del aprendizaje profundo, un codificador puede definirse como una función f parametrizada por θ que transforma una entrada $x \in \mathbb{R}^d$ en una representación latente $h \in \mathbb{R}^q$, con $q < d$, de acuerdo con la expresión [51, 52]:

$$h = f_{\theta}(x) \quad (2.17)$$

La representación h se denomina *embedding* o representación latente, y captura los factores explicativos más relevantes de los datos de entrada en un espacio de dimensión inferior [52].

Un decodificador es la función simétrica al codificador: a partir del espacio latente y parametrizada por ϕ , produce una reconstrucción r de la entrada [52]:

$$r = g_{\phi}(h), \quad (2.18)$$

Vision Transformer (ViT)

Un transformador [53] es una arquitectura basada en el mecanismo de autoatención que, a diferencia de las CNN, captura dependencias globales entre todos los elementos de una secuencia sin restricciones de localidad. Introducido originalmente para traducción automática, su capacidad de modelar relaciones a larga distancia lo ha convertido en la base de numerosas arquitecturas de aprendizaje profundo modernas.

El ViT [33] es una arquitectura de aprendizaje profundo que adapta el mecanismo de *transformer*, originalmente diseñado para procesamiento de lenguaje natural, al dominio de la imagen.

Para ello, una imagen de entrada $x \in \mathbb{R}^{H \times W \times C}$ se divide en $N = HW/P^2$ parches 2D aplanados $x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$, siendo (H, W) la resolución de la imagen original, C el número de canales y (P, P) la resolución de cada parche; este número de parches coincide, a su vez, con la longitud efectiva de la secuencia que recibe el transformador.

Cada parche se proyecta después a una dimensión D mediante una capa lineal, dando lugar a los *embeddings* de parche que servirán de entrada al modelo.

Antes de introducir esos *embeddings* en el codificador, se les añaden codificaciones posicionales que preservan su orden dentro de la imagen original. La secuencia resultante se procesa entonces mediante capas alternas de autoatención multi-cabeza, que capturan las relaciones entre todos los parches de forma global, y bloques de red neuronal de alimentación directa, que aplican una transformación no lineal independiente a cada posición de la secuencia; ambos bloques constituyen, en conjunto, el codificador del transformer [33].

Atención cruzada

Dada una entrada formada por una consulta y una pareja clave-valor, el mecanismo de atención asigna esa entrada a una salida, lo que le permite modelar dependencias entre elementos de las secuencias de entrada o salida con independencia de la distancia que los separe [53]. Concretamente, la consulta se utiliza para buscar información relevante, mientras que las claves son los elementos con los que se compara la consulta en búsqueda de una correspondencia, y los valores representan el contenido asociado a cada clave.

La salida se obtiene calculando una función de compatibilidad entre la consulta y cada clave, que determina el peso asignado al valor correspondiente; los valores así ponderados se suman finalmente para producir la salida [53].

La similitud entre cada consulta y cada clave determina el grado de atención que recibe cada valor, y la salida se calcula como:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (2.19)$$

donde d_k es la dimensión de las claves, y el factor $\sqrt{d_k}$ escala el producto para evitar que los valores sean demasiado grandes antes de aplicar la función *softmax* [53].

En la autoatención, Q , K y V provienen de la misma secuencia, lo que permite que cada elemento atienda al resto de elementos de su propia secuencia. La atención cruzada [53] es la variante en la que Q proviene de una secuencia y K y V provienen de otra distinta, permitiendo que dos representaciones de naturaleza diferente interactúen e intercambien información entre sí.

MedSAM [50] es una adaptación del modelo SAM [54] al ámbito de la imagen médica, motivada por la ausencia de un modelo universal de segmentación médica, es decir, un modelo que no requiera entrenamiento adicional para aplicarse sobre nuevas tareas de segmentación [50]. Para construirlo, se aplica un *fine-tuning* de SAM sobre un gran conjunto de datos sin precedentes, compuesto por pares imagen-máscara de varias modalidades médicas [50].

Su arquitectura consta de tres componentes principales: un codificador de imagen, un codificador de *prompts* y un decodificador de máscaras [50, 54]. El primero, basado en un transformador preentrenado, extrae las características de la imagen de entrada. El codificador de *prompts* [50], por su parte, recibe la indicación proporcionada por el usuario (cajas delimitadoras) y la transforma en una representación vectorial apta para el procesamiento. Finalmente, el decodificador de máscaras combina las

salidas de ambos codificadores mediante atención cruzada para generar la máscara de segmentación final [50].

METODOLOGÍA

Este capítulo justifica las decisiones de diseño adoptadas en el trabajo y explica algunos aspectos importantes de la implementación. El flujo metodológico comienza contextualizando el conjunto de datos, sus características y limitaciones. A partir de ahí, se introduce la segmentación y se detalla el diseño experimental, con los modelos seleccionados, su justificación y la estrategia de evaluación sobre imágenes y parches. Por último, se describen las tareas de clasificación, la extracción de parches, la gestión del desequilibrio de clases, las métricas, la búsqueda de hiperparámetros, la visualización del espacio de características, los *pipelines* y las medidas de reproducibilidad.

3.1. Entorno de ejecución

El proyecto se ha desarrollado en Python, un lenguaje consolidado como referencia para tareas de inteligencia artificial gracias a dos características complementarias, entre otras. Por un lado, ofrece una amplia variedad de librerías de código abierto que cubren todo el flujo de trabajo, desde el preprocesamiento de los datos hasta el entrenamiento de los modelos. Por otro, sus rasgos como lenguaje interpretado, de alto nivel, de propósito general y multiplataforma facilitan tanto el desarrollo como la portabilidad del código. Las versiones utilizadas son Python 3.10 y PyTorch 2.10.

La ejecución se ha distribuido entre dos entornos. La segmentación se ha realizado en local, sobre un portátil ASUS F15 con unidad de procesamiento gráfico (GPU, por sus siglas en inglés) GeForce RTX 3050 (4 GB de VRAM) y CUDA 12.7, una configuración suficiente para esta tarea. El resto del flujo experimental se ha ejecutado en el servidor de la Universitat de les Illes Balears, equipado con una tarjeta NVIDIA GeForce RTX 3060 (aproximadamente 12 GB de VRAM) y CUDA 13.2.

3.2. Conjunto de datos

El conjunto de datos utilizado en este TFG ha sido proporcionado por el Grupo de Investigación en Litiasis Renal de la Universitat de les Illes Balears, que lleva recopilando cálculos renales desde finales de los años ochenta y ha acumulado hasta la fecha un total aproximado de 22 000 muestras. Esta línea de trabajo ha dado lugar al Biobanc de Càlculs Renals de la Universitat de les Illes Balears (BICUIB), creado para la gestión sistemática del material recopilado y su puesta a disposición en el ámbito de la investigación biomédica.

Las imágenes que componen el conjunto se han adquirido con una cámara digital del modelo CMEX-10PRO, que proporciona dos resoluciones distintas: 1792×1372 y 3583×2746 píxeles, y están guardadas en formato JPG, a excepción de algunas imágenes conservadas en TIF, un formato sin pérdidas que preserva íntegramente la calidad original de la captura.

En todos los casos, las muestras se han fotografiado sobre un fondo milimetrado con una separación de 1 mm entre líneas, cosa que permite verificar visualmente la escala del cálculo.

Las imágenes etiquetadas disponibles para este trabajo se encuentran agrupadas en 9 categorías (carpetas) que siguen la nomenclatura y los criterios de clasificación propuestos por Grases *et al.* [3].

Como paso previo al diseño experimental, se ha sometido el conjunto de datos a un proceso de preparación para organizar las imágenes en función del tipo de vista del cálculo (superficie y sección) y descartar aquellas muestras o clases cuya inclusión podía comprometer el aprendizaje del modelo. En aquellos casos en los que una misma imagen contenía fragmentos pertenecientes a ambas vistas, se ha procedido a duplicar la imagen, y durante la fase de segmentación se genera una máscara por cada imagen que contiene únicamente los fragmentos pertenecientes a una vista u otra.

A continuación, se han descartado dos de las nueve clases originales: La clase 10 por contar con muestras insuficientes (6 imágenes), ya la 2b por presentar rasgos visuales prácticamente coincidentes con los de la clase 2a; las diferencias entre ambas solo son perceptibles bajo microscopio óptico, por lo que no son reconocibles a partir de las imágenes empleadas en este trabajo.

Por otro lado, las subclases 1aI+COD y 2a+COD se han fusionado con las clases 1aI y 2a respectivamente. Estas subclases agrupan cálculos de tipo 1aI y 2a que presentan depósitos superficiales de COD; dado que el núcleo y la composición principal coinciden con los de las clases originales y que, además, contaban con muy pocas muestras como para sostener una categoría propia, su tratamiento como clases independientes habría introducido fragmentación innecesaria en el conjunto de datos sin un beneficio claro para la clasificación.

Asimismo, dentro de las clases conservadas se han eliminado de imágenes que se engloban dentro de uno de los siguientes escenarios:

- Imágenes con un nivel de zoom excesivo, que impedía contextualizar el cálculo respecto al patrón milimetrado del fondo y alteraba la escala respecto al resto de imágenes.
- Imágenes correspondientes a subtipos identificados como poco frecuentes dentro de su clase (particularmente en las clases 1aI y 4I-4II), y que podían introducir

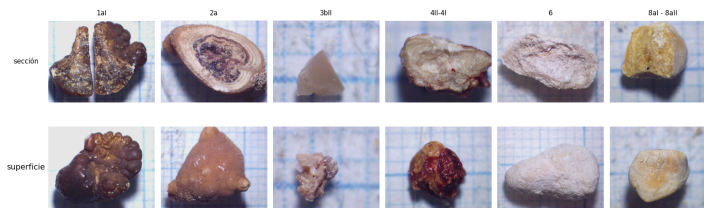


Figura 3.1: Muestras de imágenes agrupadas por clase y vista.

Tabla 3.1: Distribución del conjunto de datos procesado por clase y vista.

Clase	Sección	Superficie	Total
1aI	27	61	88
2a	64	40	104
3aII	2	22	24
3bII	33	27	60
4II-4I	39	16	55
6	26	9	35
8aI-8aII	20	11	31
Total	211	186	397

cierto ruido en el aprendizaje del modelo.

Cabe recalcar que el conjunto de datos presenta una limitación en cuanto a separación de las clases: la distinción entre las clases 3bII y 4I-4II presenta cierto grado de subjetividad, ya que ambas comparten rasgos morfológicos similares en algunas muestras.

Tras este proceso de preparación y limpieza de datos, el conjunto de datos final consta de 7 clases y un total de 397 imágenes distribuidas de manera desproporcionada entre clases y entre vistas (tabla 3.1).

Por último, hay que tener presente que tan solo un porcentaje reducido de cálculos dispone de imágenes de ambas vistas. Esta limitación constituye la principal razón por la cual no se ha explorado la vía de fusión de las características de las imágenes de dos vistas de un cálculo en un único vector (en el caso del XGBoost) para evaluar el rendimiento del modelo tal como se ha abordado en los trabajos del grupo López *et al.* [4].

3.2.1. Caracterización estadística del conjunto

Para dar una visión más detallada de la distribución de los cálculos en las imágenes, se ha realizado un análisis estadístico previo del conjunto. Las imágenes con las que se trabaja presentan una variabilidad notable con respecto al tamaño del cálculo y el número de fragmentos que lo componen. Concretamente, clases como 1aI o 3aII contienen mayoritariamente cálculos de mayor tamaño, relativamente, y compuestos por un único fragmento. En cambio, clases como 2a, 3bII, 4I-4II, 6 y 8aI-8aII agrupan varios cálculos formados por fragmentos pequeños. La figura 3.2 caracteriza esta viabilidad

3. METODOLOGÍA

indicando para cada imagen el número de fragmentos que contiene, el porcentaje que ocupa el cálculo y los tamaños de los fragmentos mayores y menores.

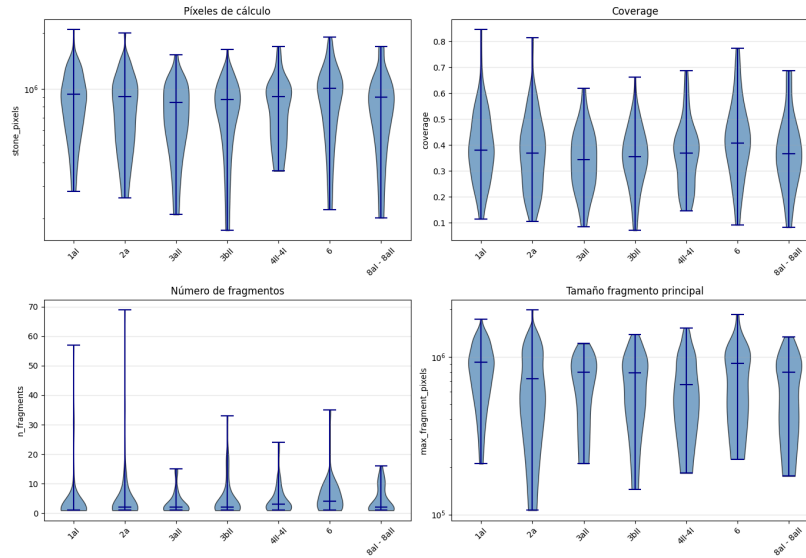


Figura 3.2: Distribución por clase del área, la cobertura, el número de fragmentos y el tamaño del fragmento principal de los cálculos.

Este diagnóstico previo aporta información cuantitativa relevante sobre cómo se distribuyen los cálculos en las imágenes:

- En mediana, el cálculo cubre el 37 % de la imagen completa, con la mayoría de los casos entre el 27 % y el 45 %.
- El número de fragmentos por imagen se sitúa entre 1 y 5 con una mediana de 2 fragmentos, aunque se observan casos aislados con un número notablemente superior (hasta 69 fragmentos), generalmente asociados a precisiones bajas del modelo de segmentación a la hora de generar las máscaras de los cálculos compuestos por varios fragmentos pequeños y solapados.
- El fragmento principal de cada cálculo tiene una mediana de $8,2 \times 10^5$ píxeles ($\approx 904 \times 904$ píxeles), lo que confirma que cada imagen aporta una región conexas de tamaño apto para la extracción de parches.

Estas estadísticas reflejan la heterogeneidad del conjunto de datos a varios niveles. El tamaño del cálculo abarca más de un orden de magnitud entre el caso más pequeño y el más grande, la cobertura del cálculo en la imagen varía significativamente entre clases, y el número de fragmentos por imagen oscila entre uno y varios. A pesar de esta variabilidad, la mayoría de las imágenes disponen de un fragmento principal con tamaño y cobertura suficientes para una clasificación a nivel de imagen y la posterior extracción de parches a partir de ellas.

Por último, los casos con un número elevado de fragmentos identificados son una señal de que las máscaras generadas no capturan bien los cálculos compuestos por varios fragmentos pequeños, hecho que se ha observado ya durante la generación de

las máscaras y que conviene tener presente a la hora de interpretar los resultados sobre las clases más afectadas por este patrón.

3.3. Segmentación

Como paso previo al entrenamiento de los modelos, se ha aplicado un proceso de segmentación a las imágenes para obtener, en cada una, una máscara binaria que aísla el cálculo del resto del fondo. Este paso es habitual en los *pipelines* de clasificación basados en imagen y resulta especialmente útil en este trabajo, ya que el fondo milimetrado no aporta información sobre la clase del cálculo y puede introducir ruido visual, sesgando el aprendizaje del modelo si se trata como parte de la representación de cada muestra.

Para esta tarea se ha seleccionado **MedSAM** [50], un modelo universal para la segmentación de imágenes médicas que no requiere *fine-tuning* adicional sobre el dominio de los cálculos renales y que funciona mediante *prompts*, de modo que, para generar la máscara binaria de un objeto, necesita como entrada la imagen y una caja delimitadora que lo ubique dentro de ella.

Aprovechando este funcionamiento, la segmentación se ha llevado a cabo mediante un flujo semiautomático: la anotación de las cajas delimitadoras se realiza manualmente con la herramienta *Label Studio*, mientras que la generación de las máscaras se delega al MedSAM.

El proceso manual consiste en importar las imágenes a *Label Studio*, anotar las cajas delimitadoras y exportar el resultado en un fichero JSON con la información asociada a cada imagen: nombre e identificador, cajas delimitadoras en formato COCO y área que ocupan, entre otros campos.

La generación de las máscaras es un proceso iterativo: si la máscara obtenida no se ajusta adecuadamente al cálculo, se corrigen las cajas delimitadoras, se genera un JSON actualizado y se repite la inferencia hasta obtener un resultado satisfactorio.

Cabe señalar que la calidad de las máscaras se ve afectada por dos factores. Por un lado, las variaciones de brillo presentes en algunas imágenes, que se manifiestan como sombras sobre el cálculo o como regiones especialmente blancas dentro de este —normalmente zonas pequeñas en la vista de sección. Por otro lado, la presencia de varios fragmentos solapados en una misma imagen, que dificulta obtener máscaras precisas.



Figura 3.3: Segmentación con MedSAM: ejemplo de una imagen con su máscara generada y la correspondiente imagen de superposición.

3.4. Diseño experimental

Siguiendo los objetivos planteados en este TFG, el diseño experimental compara los enfoques SL y DL para la tarea de clasificación de cálculos renales, e introduce, además, un enfoque híbrido que combina elementos de ambos.

3.4.1. Modelos seleccionados

Los modelos elegidos como representantes de cada enfoque son XGBoost y ResNet50, ambos consolidados como parte del estado del arte en este ámbito.

Como descriptores de entrada para XGBoost se han empleado el LBP e histogramas de HSV. Aunque la línea metodológica de López-Tiro *et al.* [4], identifica la combinación de LBP + energías del HSV como la que mejor separabilidad ofrece, sus resultados derivan de experimentos realizados íntegramente sobre parches. Por ello, ya que se trabaja en este TFG con imágenes, además, se ha optado por esa decisión, y la inclusión de los histogramas basados en energía se añaden como parte de estudios de ablación.

Los parámetros concretos del LBP (P, R) —el número de puntos vecinos y el radio sobre el que se calcula el patrón binario— condicionan la escala de textura que se captura. En la literatura, el par (8, 1) es la elección estándar, ya que ofrecen simplicidad y permite adecuadamente texturas finas. En este trabajo se evalúa empíricamente, junto al estándar, una configuración alternativa con un radio y número de puntos mayores (8, 2), (16, 1) y (16, 2) con el fin de cubrir también texturas a escala intermedia. La elección final se realiza comparando el rendimiento de XGBoost sobre la tarea binaria con cada una de las cuatro combinaciones.

En cuanto al aprendizaje profundo, ResNet50 ha sido evaluado en numerosos trabajos del dominio, donde se ha comparado tanto con modelos de SL como con otras CNN y modelos basados en *transformers* (ViT) [32]. Frente a otras familias como Inception, ResNet50 ofrece una arquitectura más sencilla y estudiada, con un compromiso parámetros-rendimiento adecuado para este tipo de tarea.

Dado el tamaño reducido del conjunto de datos, se ha adoptado una estrategia de *transfer learning*: el *backbone* de ResNet50 se inicializa con los pesos preentrenados sobre ImageNet y se mantiene congelado durante el entrenamiento, de modo que únicamente se entrena la cabecera de clasificación que se modifica para adaptar el número de salidas del modelo al número de clases deseadas según el tipo de clasificación.

3.4.2. Modelo híbrido: *HybridResNet50*

Los resultados prometedores de ambos enfoques, SL con XGBoost y DL con ResNet50, ha motivado la idea de estudiar el efecto de la combinación de las características extraídas de ambos modelos, construyendo un modelo híbrido llamado HybridResNet50. Concretamente, se combinan los descriptores de textura y color (LBP + HSV) juntamente con las características aprendidas por el *backbone* del ResNet50, mediante fusión temprana por concatenación: el descriptor manual, de 40 dimensiones, se precálcula sobre la región segmentada del cálculo y se concatena con el vector de 2048 dimensiones de la salida del *backbone*, produciendo un vector de 2088 dimensiones que alimenta a la capa de clasificación. De este modo, es posible atribuir cualquier

diferencia de rendimiento entre ambos modelos a la incorporación del descriptor manual.

La hipótesis a contrastar es si la información estructurada y específica del dominio que aportan LBP y HSV produce una mejora sobre la representación aprendida por la red, o si esta última ya captura implícitamente la misma información durante el entrenamiento.

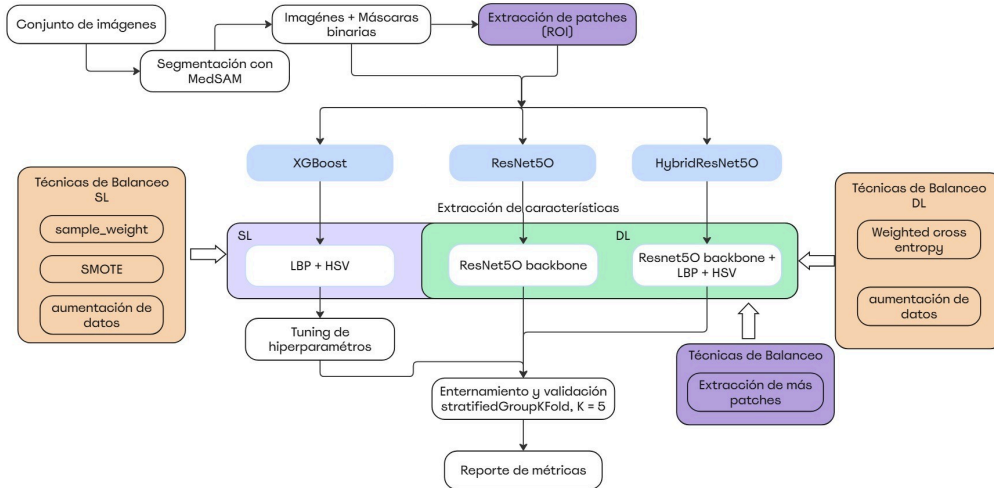


Figura 3.4: Diagrama general del pipeline experimental que ilustra los modelos usados con los dos tipos de entrada y las técnicas de balanceo usadas en cada escenario.

3.4.3. Imágenes completas frente a parches

La decisión de comparar la clasificación sobre imágenes completas y sobre parches extraídos responde a las dos perspectivas que conviven en los trabajos de referencia. Por un lado, el *pipeline* habitualmente adoptado en los trabajos del estado del arte recomienda trabajar con parches, ya que permiten aumentar el número de muestras por clase y emulan en cierto modo el procedimiento del MCA, en el que el especialista examina regiones específicas del cálculo en lugar de su totalidad. Por otro lado, algunos trabajos advierten que la clasificación basada únicamente en parches puede introducir un sesgo favorable sobre las métricas reportadas y recomiendan evaluar también el rendimiento sobre imágenes [16]. Ante esta doble perspectiva, en este TFG se han realizado ambos experimentos y se han comparado sistemáticamente sus resultados, lo que permite además verificar empíricamente cuál de los dos enfoques resulta más adecuado para este problema.

3.5. Tareas de clasificación

En este TFG se consideran dos tareas de clasificación complementarias: una **binaria** y otra **multiclase**.

La clasificación binaria consiste en distinguir entre los cálculos de tipo COM y el resto del conjunto. La clase COM agrupa los subtipos 1aI y 2a, dos variantes de COM con etiologías diferenciadas (papilar y de cavidad).

La clasificación multiclase agrupa los cálculos en cinco tipos clínicamente diferenciados: COM (1aI, 2a), COD (3aII), Mixto (3bII, 4II-4I), Infeccioso (6) y Úrico (8aI-8aII). Esta agrupación se alinea con la clasificación utilizada por el Grupo de Investigación en Litiasis Renal de la Universitat de les Illes Balears.

La binaria permite identificar específicamente los cálculos COM, que son los más frecuentes en general [3], constituyendo un punto de partida con una simplicidad relativa, dada por la diferencia visual entre ambas clases. La clasificación multiclase, en cambio, es la que aporta el valor diagnóstico real del trabajo, ya que el tratamiento médico y la estrategia de prevención de la recurrencia varían según el tipo concreto de cálculo identificado. Sumado a esto, este tipo de clasificación el formato estándar de evaluación en los trabajos de referencia del estado del arte.

3.6. Extracción de parches

Dadas las posiciones no uniformes de los cálculos dentro de la imagen, y aprovechando que se dispone de las máscaras calculadas en la fase de segmentación, se ha optado por una estrategia de extracción de parches basada en regiones de interés (*ROI*). La disponibilidad de las máscaras hace que el cálculo del centroide sea inmediato; al mismo tiempo, este enfoque permite maximizar la información útil de cada parche y reduce la varianza entre los parches extraídos de un mismo cálculo.

Como parámetros del proceso, se exige una cobertura mínima del 90 % de cálculo dentro de cada parche y se utiliza un desplazamiento igual a la mitad del tamaño del parche, lo que permite un solapamiento máximo del 50 % entre parches contiguos.

3.6.1. Algoritmo de extracción de parches

Este algoritmo genera un número definido de parches y sus máscaras con un tamaño determinado y una distancia mínima de solapamiento entre ellos.

En primer lugar, el algoritmo identifica las regiones de interés, es decir, los píxeles blancos de la máscara binarizada que pertenecen al cálculo, y los almacena en una lista. A continuación, filtra aquellos centros ubicados en zonas perimetrales para asegurar que ningún parche exceda los límites de la imagen.

En cada iteración, se selecciona aleatoriamente un centro candidato y se construye un parche a su alrededor. Si el parche generado cumple con la cobertura mínima y la distancia mínima respecto a los parches ya creados, se almacena en la lista definitiva. Este procedimiento se repite hasta alcanzar el número objetivo de parches, agotar el número máximo de intentos o consumir por completo la lista de puntos candidatos.

Cabe recalcar, que este enfoque no garantiza obtener la cantidad solicitada de parches, ya que el resultado final depende principalmente del área ocupada por el cálculo, del tamaño del parche y de la distancia establecida entre los centros de los parches.

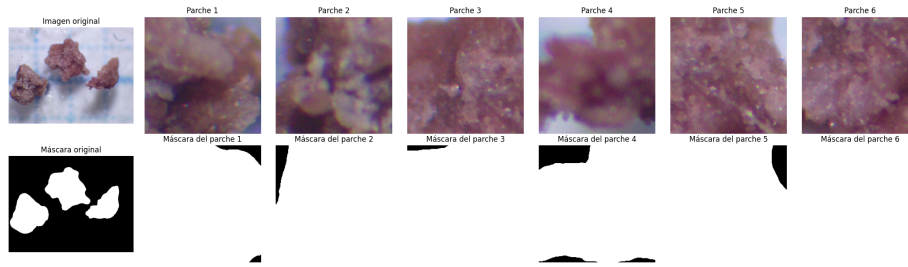


Figura 3.5: Ejemplo de parches y sus máscaras extraídos a partir de una imagen con cobertura mínima del 90 % y con tamaño de 224 x 224.

3.6.2. Selección del tamaño del parche

La elección del tamaño de parche debe ser adecuada para los dos enfoques. En el caso del XGBoost no existe un tamaño de entrada concreto y obligatorio, ya que el modelo no procesa la imagen directamente, sino un vector de características extraídas (LBP + HSV); ResNet50, en cambio, trabaja con un tamaño de entrada fijo de $224 \times 224 \times 3$ píxeles, por lo que este valor se establece como cota inferior del rango evaluado, con el fin de evitar un redimensionamiento previo. A partir de ahí, y con un salto de 32 píxeles, se definen los tamaños candidatos restantes: 256×256 , 288×288 y 320×320 .

La cota superior del rango responde a dos motivos. Por un lado, la literatura muestra que aumentar el tamaño del parche no reporta mejoras ilimitadas, ya que López *et al.* [4] evaluaron tamaños de hasta 512×512 y observaron que, si bien mejoraban ligeramente el rendimiento de los modelos superficiales, inducían sobreajuste en los profundos, razón por la cual optaron por un tamaño moderado de 256×256 . Por otro lado, las dimensiones de las regiones de interés del propio conjunto de datos limitan el tamaño máximo aplicable, puesto que los parches más grandes no pueden extraerse de todas las imágenes. Dentro de este rango, el mejor tamaño se determina de forma empírica, evaluando los clasificadores XGBoost y ResNet50 sobre la tarea binaria para cada uno de los tamaños candidatos.

3.6.3. Agregación de predicciones a nivel de imagen

Cuando los modelos se entrenan y se evalúan sobre parches, el resultado del clasificador es una predicción por cada parche y no una predicción única por imagen, por lo que, para obtener una etiqueta final comparable con la del escenario sobre imágenes completas, las predicciones de los parches de una misma imagen se agregan mediante el promedio de sus probabilidades: se calcula la media de las probabilidades de pertenencia a cada clase predichas sobre todos los parches de la imagen y se asigna a esta la clase con probabilidad media máxima.

3.7. Gestión del desequilibrio de clases

El conjunto de datos presenta un desequilibrio notable entre clases, lo que motiva el uso de estrategias específicas para evitar que el aprendizaje quede sesgado hacia las clases mayoritarias. Las técnicas consideradas varían según el tipo de entrada (imagen o parche) y según el modelo (SL o DL), ya que la representación de los datos y la

naturaleza del entrenamiento condicionan qué enfoques son aplicables y eficaces en cada caso.

3.7.1. Caso de imágenes

XGBoost

Se han considerado tres estrategias. La primera es la utilización del parámetro *sample_weight*, que permite asignar a cada muestra un peso inversamente proporcional a la frecuencia de su clase, de modo que la contribución total de cada categoría a la función de pérdida quede equilibrada. La segunda es la aplicación de aumentación de datos sobre las imágenes antes de la extracción de características, mediante transformaciones suaves (geométricas y fotométricas) que generan nuevas muestras. La tercera es la aplicación de una técnica de sobremuestreo sintético de la clase minoritaria (SMOTE, por sus siglas en inglés) [55] muy utilizada en el campo del aprendizaje automático para el balanceo de clases [56, 57]. Esta, actúa sobre el espacio de los vectores de características ya extraídos y genera muestras sintéticas mediante interpolación entre vectores existentes en el caso de las clases minoritarias.

ResNet50

Para el aprendizaje profundo se usa principalmente una función de pérdida ponderada (*Weighted Cross-Entropy*), que penaliza con mayor peso los errores cometidos sobre las clases minoritarias durante el entrenamiento.

3.7.2. Caso de parches

XGBoost

La estrategia de balanceo se integra directamente en el proceso de extracción de parches: para las clases minoritarias, se relaja el umbral de cobertura mínimo exigido sobre el fragmento y se generan parches adicionales aleatorios, concentrados en las regiones de interés, hasta alcanzar el número objetivo por clase. Este enfoque permite equilibrar la representación de cada clase en el conjunto de entrenamiento sin recurrir a interpolaciones sintéticas o transformaciones del descriptor.

ResNet50

Los parches adicionales se combinan con el uso de la función de pérdida ponderada (*Weighted Cross-Entropy*) para intentar solucionar el problema del desbalance en este modelo.

3.8. Métricas de evaluación

Para evaluar el rendimiento de un modelo resulta útil apoyarse en la matriz de confusión, una representación visual de los aciertos y errores cometidos por el clasificador. Se trata de una tabla de tamaño 2×2, en el caso de una clasificación binaria, en la que las filas indican la clase real de cada muestra y las columnas indican la clase predicha por el modelo. De este modo, cada celda representa el número de muestras

correspondientes a una de las cuatro combinaciones posibles entre la predicción y la etiqueta real: verdaderos positivos (TP), falsos negativos (FN), falsos positivos (FP) y verdaderos negativos (TN).

Los TP y TN cuantifican las muestras correctamente clasificadas de las clases positiva y negativa, respectivamente. Por otro lado, los FN y FP corresponden a los casos en los que el modelo comete un error de asignación; concretamente, al etiquetar una muestra positiva como negativa (FN) o una negativa como positiva (FP).

A partir de estas cuatro cantidades se construye el resto de métricas que se describen a continuación [58, 59].

Exactitud: mide la proporción total de las muestras correctamente clasificadas.

$$\text{Exactitud} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (3.1)$$

Esta métrica por sí sola resulta insuficiente porque, por un lado, puede proporcionar valores altos en conjuntos de datos desbalanceados aunque el clasificador solamente haya aprendido a predecir la clase mayoritaria y, por otro, asigna el mismo peso a los falsos negativos (FN) y falsos positivos (FP).

Por ello resulta imprescindible usar métricas que permiten medir correctamente el rendimiento del modelo a la hora de clasificar muestras de la clase positiva (precisión y exhaustividad).

Precisión: es una medida de exactitud de la clasificación de muestras positivas, ya que proporciona la cantidad de las muestras positivas etiquetadas correctamente sobre el total de las muestras etiquetadas como positivas.

$$\text{Precisión} = \frac{TP}{TP + FP}, \quad (3.2)$$

Exhaustividad: conocida también como tasa de verdaderos positivos (*True Positive Rate, TPR*), es una métrica de completud que calcula el porcentaje de las muestras de la clase positiva que fueron correctamente etiquetadas.

$$\text{Exhaustividad} = \frac{TP}{TP + FN}, \quad (3.3)$$

F1-score: equilibra el peso de los falsos negativos y falsos positivos mediante la media armónica entre precisión y exhaustividad. Solo alcanza valores elevados cuando ambas componentes lo son, ya que basta con que una de ellas sea baja para que el *F1-score* también lo sea. Es una métrica idónea para tareas de clasificación en las que el interés se centra en la clase positiva, como es el caso de la clasificación de COM vs. Otros.

$$\text{F1-score} = 2 \cdot \frac{\text{Precisión} \cdot \text{Exhaustividad}}{\text{Precisión} + \text{Exhaustividad}}, \quad (3.4)$$

ROC-AUC: la curva ROC (*Receiver Operating Characteristic*) visualiza el comportamiento del modelo a lo largo de distintos umbrales de decisión. Se construye representado en el eje vertical la tasa de verdaderos positivos (TPR), y en el horizontal la tasa de falsos positivos (FPR = FP/(FP+TN)), donde cada umbral define un punto en la curva. Así, un modelo perfecto es aquel que puede alcanzar, en algún umbral, una TPR de 1.0 con una FPR de 0.0.

El área bajo la curva (AUC, por sus siglas en inglés) representa la probabilidad de que el modelo asigne una puntuación mayor a una muestra positiva elegida al azar que a una negativa [60].

3.9. Validación cruzada

Como técnica de evaluación se ha empleado la validación cruzada por la robustez estadística que aporta [41]. En cada *fold* se entrena un modelo distinto sobre un subconjunto de datos y se evalúa sobre el subconjunto restante, de manera que se aprovecha la totalidad de las muestras para la evaluación. Esto permite obtener una estimación del rendimiento más estable y con menor varianza que la que ofrecería una única partición *train/test*.

Al iterar el entrenamiento y la evaluación en diferentes subconjuntos (*fold*s), el rendimiento final reportado no depende de una división aleatoria particular, garantizando una evaluación estadísticamente más estable y objetiva. La estrategia concreta es *StratifiedGroupKFold*, una variante que combina dos propiedades imprescindibles en este trabajo: la estratificación, que preserva la proporción de cada clase a lo largo de todos los *fold*s, y la agrupación, que asegura que todas las muestras pertenecientes a un mismo grupo permanezcan dentro del mismo *fold* (entrenamiento o *test*).

3.9.1. Agrupación de imágenes por cálculo

Para usar la técnica de *StratifiedGroupKFold* es necesario asignar identificadores únicos a los cálculos para poder agrupar, bajo el mismo identificador, las imágenes que les pertenecen. Este proceso aprovecha la nomenclatura de las imágenes para extraer esos identificadores, ya que estas siguen un patrón que permite determinar unívocamente un cálculo dentro de su clase, así como las imágenes que le pertenecen. Esta asignación resulta necesaria también en la generación de los parches, puesto que estos deben mantener el identificador de la imagen de la cual provienen para evitar la fuga de datos y poder, posteriormente, calcular las métricas de clasificación agregadas por imagen.

La agrupación, en el presente trabajo, es relevante ya que la fuga de información entre subconjuntos podría producirse por dos vías distintas: por un lado, un mismo cálculo puede aparecer fotografiado en sus dos vistas, superficie y sección; y, por otro, un mismo cálculo y vista pueden estar representados por varias imágenes capturadas a distintas distancias focales. En ambos casos, todas las imágenes derivadas de un mismo cálculo deben pertenecer al mismo *fold*, para evitar que el conjunto de *test* contenga información correlacionada con el de entrenamiento.

3.10. Búsqueda de hiperparámetros

XGBoost dispone de un número elevado de hiperparámetros que condicionan su rendimiento de forma significativa. Para acotar la búsqueda, se ha seleccionado un subconjunto de los más influyentes, detallados en la tabla 3.2, y se ha realizado la optimización sobre el espacio definido por estos parámetros.

Como *framework* de optimización se usa **Optuna** [61], una herramienta ampliamente adoptada en trabajos recientes que, a diferencia de las estrategias clásicas como la búsqueda exhaustiva o aleatoria, permite una exploración más eficiente del espacio de hiperparámetros basándose en optimización bayesiana: en lugar de evaluar combinaciones de manera uniforme o aleatoria, determina las configuraciones que tienen mayor probabilidad de mejorar el rendimiento del modelo y concentra su búsqueda en ellas.

La búsqueda bayesiana se aplica a XGBoost dentro de cada *fold* de la validación cruzada. Los rangos explorados para cada hiperparámetro se recogen en la tabla 3.2.

Hiperparámetro	Rango
Profundidad máxima	[3, 10]
Tasa de aprendizaje	[0,01,0,3] (escala log)
Número de estimadores	[100,600]
Peso mínimo del nodo hijo	[1, 10]
Proporción de submuestreo	[0,5, 1,0]
Proporción de características por árbol	[0,3, 1,0]

Tabla 3.2: Espacio de búsqueda de Optuna para los hiperparámetros de XGBoost.

La función objetivo recibe el conjunto de entrenamiento del *fold* externo, lo divide en *folds* internos, entrena XGBoost con la combinación de hiperparámetros propuesta por Optuna y devuelve la media del *F1* ponderada sobre los *folds* internos, que es la métrica que Optuna maximiza.

Una vez finalizado el proceso, los mejores hiperparámetros se utilizan para entrenar el modelo final sobre todo el conjunto de entrenamiento externo, que después se evalúa sobre el conjunto de test del *fold*.

3.11. Visualización del espacio de características

Para interpretar el rendimiento de los modelos más allá de las métricas agregadas, se ha visualizado la distribución de las clases en el espacio de características de cada modelo mediante Aproximación y Proyección Uniforme de Variedades (UMAP, por sus siglas en inglés) [62], una técnica de reducción de dimensionalidad no lineal que proyecta los datos a un espacio de menor dimensión preservando tanto la estructura local como la estructura global de los datos.

La proyección se aplica sobre representaciones distintas según el modelo. En el caso de XGBoost, se proyectan directamente los descriptores manuales LBP+HSV de 40 dimensiones. En el caso de la ResNet50 y del modelo híbrido, se proyectan las representaciones aprendidas de 256 dimensiones, extraídas como la entrada a la última capa lineal de clasificación mediante un *hook* sobre dicha capa. Cabe señalar que, por simplicidad, las representaciones se han extraído a partir de los modelos entrenados en un único *fold* de la validación cruzada, lo que confiere a estas visualizaciones un carácter ilustrativo.

3.12. *Pipelines*

Los *pipelines* de los tres modelos concentran el flujo completo de cada experimento, desde la división en *folds* hasta la agregación de los resultados. A continuación se explica cada uno de ellos.

3.12.1. Pipeline del XGBoost

A diferencia de los modelos basados en CNN, las características usadas en el XGBoost, al ser deterministas, se extraen una sola vez en la inicialización. El flujo que le sigue se divide en dos fases: la búsqueda bayesiana de hiperparámetros con Optuna, y la evaluación por validación cruzada. En esta segunda fase, para cada *fold* se aplica, si corresponde, una de las técnicas de balanceo (la aumentación de datos sobre las clases minoritarias, SMOTE o la ponderación por clase), y se entrena el modelo con los hiperparámetros óptimos encontrados en la primera fase.

3.12.2. Pipeline de la ResNet50

Para cada *fold* se instancia el modelo *ResNet50Classifier* con el *backbone* preentrenado en ImageNet y congelado, y la capa de clasificación final adaptada a la tarea. Concretamente, se sustituye la capa original por una cabecera que proyecta el vector de 2048 dimensiones del *backbone* a 256 mediante una capa lineal, seguida de *batch normalization*, activación *ReLU* y *dropout*, y finalmente a una última capa lineal que produce las salidas correspondientes al número de clases. Además, si es el caso de la clasificación multiclase, se calculan los pesos por clase para la *Weighted Cross-Entropy*.

3.12.3. Pipeline del HybridResNet50

Este pipeline reutiliza todo el flujo de la ResNet50 y añade el tratamiento de los descriptores manuales que alimentan la segunda rama del modelo. Estos se extraen una sola vez en la inicialización, y se almacenan en un diccionario para su recuperación durante el entrenamiento. La instanciación construye un *HybridResNet50* cuya cabecera acepta el vector concatenado de la representación profunda y los descriptores manuales.

En este enfoque, como que ambas ramas operan sobre fuentes de datos distintas (las imágenes que procesa la red y los vectores ya extraídos), es necesario un emparejamiento correcto de la información que recibe el modelo híbrido al procesar cada muestra. Esto se implementa, construyendo una clave única (*match_key*) que forma un identificador único tanto al trabajar con imágenes como con parches. Durante el entrenamiento, el *KidneyStoneDataset* recupera el descriptor correspondiente a cada imagen a través de su *match_key*, lo que asegura que ambas ramas reciban correctamente la información de la misma muestra por las dos ramas.

3.12.4. Evaluación de los modelos

Los tres *pipelines* comparten el mismo procedimiento de evaluación. Una vez recorridos los cinco *folds*, las métricas se agregan y se guardan junto con la información necesaria para la posterior construcción de métricas por clase. Además, cuando la

entrada son parches, se calculan también las métricas agregadas a nivel de imagen mediante *soft voting*, promediando las probabilidades de los parches de cada imagen.

3.13. Reproducibilidad

La reproducibilidad de los experimentos constituye un requisito esencial para garantizar que los resultados obtenidos puedan ser replicados en otros entornos y, por tanto, que las métricas reportadas sean fiables.

Las principales fuentes de aleatoriedad en el *pipeline* empleado provienen de la división de los datos en *folds*, del orden de carga y procesamiento de las imágenes durante el entrenamiento, y de los componentes internos de los modelos que utilizan inicialización aleatoria. Para mitigar esa aleatoriedad, se fija una semilla común en todos los puntos del código donde la implementación lo permite.

Esta configuración mínima ayuda a dar mayor fiabilidad a los resultados. No obstante, conviene matizar que la aleatoriedad no puede eliminarse por completo en *pipelines* que emplean GPU, por lo que se asume la pequeña varianza residual atribuible al no-determinismo intrínseco de las operaciones en GPU.

Por último, para facilitar la trazabilidad de los resultados obtenidos y permitir la consulta del código fuente, se ha empleado *Git* para alojar el proyecto en un repositorio público¹ de *GitHub* que recoge la estructura del proyecto, describe la función de cada módulo y detalla los pasos necesarios para ejecutar los experimentos.

¹<https://github.com/Ti-trit/renal-calculi-classification>

RESULTADOS Y DISCUSIÓN

En este capítulo se presentan los resultados de los experimentos realizados: el ajuste de los parámetros (P, R) del LBP, la selección del tamaño de parche, y la evaluación de los tres clasificadores en las tareas binaria y multiclase, incluyendo las técnicas de balanceo aplicadas, bajo los dos tipos de entrada: imágenes y parches.

El rendimiento de los modelos se mide con las métricas habituales en clasificación —exactitud, precisión, exhaustividad y $F1$ —, todas en su versión ponderada por el soporte de cada clase, junto con el AUC, que evalúa la capacidad de ordenamiento de las clases con independencia del umbral de decisión.

Para la interpretación de los resultados se recurre a proyecciones UMAP del espacio de características para observar de forma cualitativa cómo se distribuyen y solapan las clases, y a las matrices de confusión, que revelan la dirección concreta de los errores. La combinación de las tres perspectivas (métricas, proyecciones y matrices) es la que sostiene el análisis a lo largo del capítulo. Finalmente, en la sección de estudio de ablación se lleva a cabo un experimento para estudiar la viabilidad de los histogramas de color usados en este trabajo frente a los basados en energías de color adoptados en los trabajos del estado del arte [4, 14].

4.1. Mejores parámetros (P, R) del LBP

El descriptor LBP depende de dos parámetros que definen cómo se muestrea la vecindad de cada píxel: el número de puntos P y el radio R . Para elegir la combinación provechosa de (P, R) para el modelo de SL se evalúan cuatro combinaciones de acuerdo con lo explicado en la sección 3.4.1. La figura 4.1 resume los resultados obtenidos para las combinaciones $(8, 1)$, $(8, 2)$, $(16, 1)$ y $(16, 2)$ evaluados con XGBoost para la clasificación binaria de imágenes.

Una primera observación de la figura 4.1 es la limitada diferencia entre las configuraciones: tanto la exactitud como el $F1$ oscilan entre el 82 % (configuraciones $[16, 1]$ y $[16, 2]$) y el 84 % (configuraciones $[8, 1]$ y $[8, 2]$), mientras que el AUC alcanza su valor

4. RESULTADOS Y DISCUSIÓN

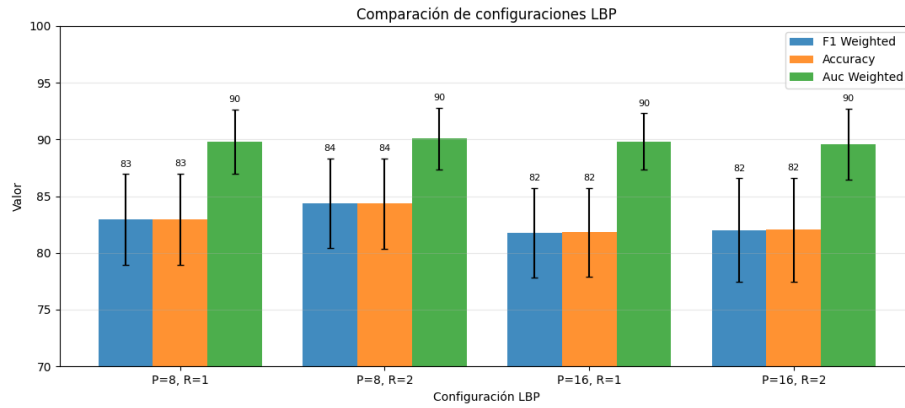


Figura 4.1: Gráfico comparativo de las métricas ponderadas AUC, exactitud y $F1$ para diferentes valores de (P, R) del XGBoost en el escenario de clasificación binaria de las imágenes.

más alto con las dos primeras configuraciones con un 90 %. Las desviaciones estándar se mantienen en el rango [3,6 %, 5,5 %] para todas las métricas y configuraciones.

La escasa diferencia observada sugiere que la información adicional aportada por las configuraciones con valores más altos de P y R resulta redundante para el XGBoost, por lo que no llega a traducirse en mejoras apreciables del rendimiento. Esta observación resulta, además, coherente con el funcionamiento del modelo descrito en la sección 2.4, puesto que, al construir cada árbol, el XGBoost selecciona en cada nodo la partición más informativa y prescinde de aquellas variables que aportan información equivalente, ya que estas no contribuyen a reducir la pérdida más allá de lo que ya logran las particiones escogidas. A este comportamiento se suma el efecto del término de regularización, que refuerza dicha tendencia al penalizar la incorporación de particiones cuya ganancia no compensa el aumento de complejidad estructural del árbol. Como consecuencia de ambos mecanismos, el modelo acaba convergiendo a un rendimiento muy similar con independencia de la configuración empleada.

Finalmente, dado que todas las configuraciones son prácticamente idénticas en exactitud y $F1$, se elige la que mejor rinde: la configuración [8,2].

4.2. Tamaño idóneo del parche

El tamaño de parche se ha evaluado sobre el escenario de clasificación binaria empleando los clasificadores XGBoost sobre descriptores manuales y la red ResNet50. En ambos casos se generaron parches de 224×224 , 256×256 , 288×288 y 320×320 píxeles, y se repitió el experimento para cada uno de ellos. Los resultados obtenidos con XGBoost se recogen en la figura 4.2, mientras que los correspondientes a ResNet50 se muestran en la figura 4.3.

En ambos experimentos, la configuración de 320×320 es la que ofrece el mejor rendimiento entre las evaluadas, si bien las diferencias respecto al resto de tamaños resultan reducidas. Con XGBoost, 320×320 alcanza valores altos de forma sistemática en todas las métricas y su ventaja en AUC supera el rango de la desviación estándar frente al resto de configuraciones; además, es la que presenta menor varianza, con un

4.2. Tamaño idóneo del parche

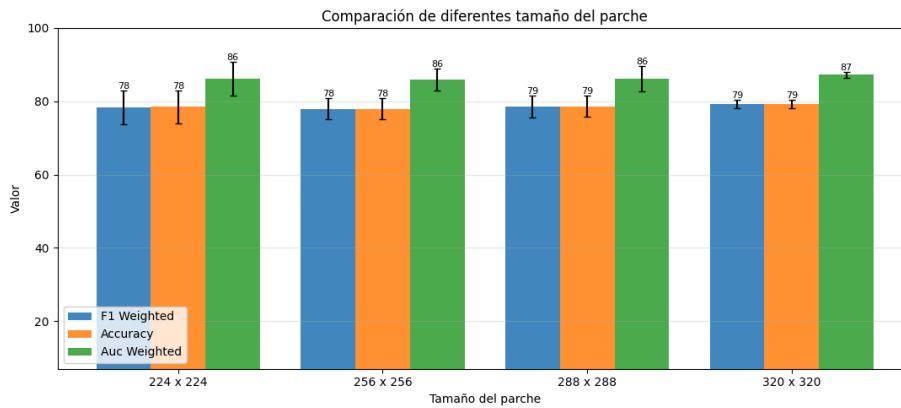


Figura 4.2: Métricas ponderadas (exactitud, *F1* y AUC) obtenidas para los diferentes tamaños de parche evaluados con el modelo XGBoost en el escenario de clasificación binaria.

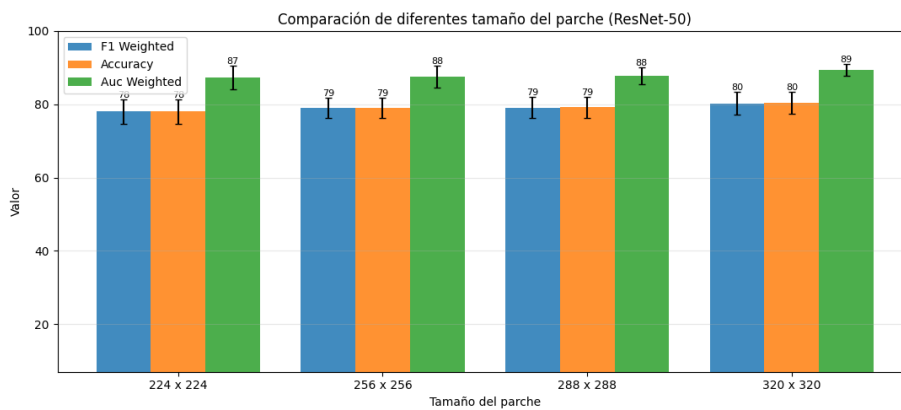


Figura 4.3: Métricas ponderadas (exactitud, *F1* y AUC) obtenidas para los diferentes tamaños de parche evaluados con el ResNet50 en el escenario de clasificación binaria.

1 % en AUC frente al 3 % y el 5 % en torno a los que oscilan los demás tamaños. Con ResNet50, las diferencias son igualmente pequeñas, aunque se aprecia una tendencia creciente: la exactitud y el *F1* pasan del 78 % con parches de 224×224 al 80 % con 320×320 , y el AUC del 87 % al 89 %. En este segundo caso, las desviaciones entre *folds*, en torno al 2-3 %, hacen que los intervalos de tamaños consecutivos se solapen, por lo que la mejora debe interpretarse como una tendencia consistente más que como una diferencia concluyente.

Aumentar el tamaño del parche conlleva un compromiso, puesto que cuanto mayor es el parche, menor es el número de parches válidos que pueden extraerse de cada imagen. El hecho de que 320×320 obtenga el mejor rendimiento en ambos modelos indica que, dentro de este rango, la ganancia asociada al mayor tamaño prevalece sobre la reducción en el número de muestras. No obstante, el mecanismo que explica dicha ganancia difiere entre uno y otro enfoque. En el XGBoost, un mayor número de píxeles permite que los histogramas de textura y color sean más estables y menos sensibles al ruido de muestreo, mientras que en ResNet50, dado que la red redimensiona todo

parche a su entrada fija de 224×224 , la ventaja no procede de una mayor resolución—que se pierde en el redimensionado—, sino del mayor contexto espacial que abarcan los parches grandes: un parche de 320×320 cubre una región más amplia del cálculo y proporciona a la red un campo de visión morfológico más grande por muestra. En ambos casos, sin embargo, se trata de interpretaciones cualitativas, ya que las diferencias observadas se sitúan dentro del margen de dispersión.

La búsqueda se acota en este tamaño porque, a 320×320 , una de las imágenes del conjunto ya queda descartada al no admitir un parche de esas dimensiones, y tamaños mayores reducirían progresivamente la cobertura del conjunto de datos. Esta acotación es coherente con lo discutido en la sección 3.6.2, donde se motivó el rango de tamaños evaluado. La coincidencia del tamaño óptimo entre ambos modelos refuerza, por tanto, la elección de 320×320 como tamaño de parche definitivo para el resto de experimentos, al no depender de las particularidades de un único enfoque. Con este parámetro fijado, junto con la configuración del LBP de la sección anterior, queda determinada la extracción de características, y el análisis puede centrarse ya en la comparación de los modelos.

4.3. Clasificación usando imágenes

Esta sección presenta los resultados obtenidos al entrenar y evaluar los tres modelos—XGBoost, ResNet50 e HybridResNet50—utilizando las imágenes de los cálculos como entrada, en las dos tareas de clasificación: la binaria y la multiclase que se recogen en la tabla 4.1. La comparación entre modelos permite, primero, identificar los de mejor rendimiento, y, segundo, valorar hasta qué punto los descriptores manuales de textura y color aportan información complementaria a la que aprende automáticamente la red profunda y si esa aportación depende de la dificultad de la tarea.

Modelo	Clasificación binaria (COM vs. Otros)					Clasificación multiclase (5 clases)				
	Exactitud	<i>FI</i>	Precisión	Exhaustividad	AUC	Exactitud	<i>FI</i>	Precisión	Exhaustividad	AUC
XGBoost	84 ± 4	84 ± 4	84 ± 4	84 ± 4	90 ± 3	70 ± 4	69 ± 4	71 ± 4	70 ± 4	88 ± 4
ResNet50	86 ± 2	86 ± 2	87 ± 2	86 ± 2	93 ± 2	70 ± 5	70 ± 4	74 ± 3	70 ± 5	90 ± 2
hybrid_ResNet50	84 ± 3	84 ± 3	85 ± 3	84 ± 3	92 ± 2	74 ± 4	74 ± 3	76 ± 3	74 ± 4	90 ± 3

Tabla 4.1: Resultados comparativos del rendimiento de los modelos en clasificación binaria y multiclase (versión ponderada) bajo *StratifiedGroupKFold* con $K = 5$ folds. Los mejores resultados de cada tarea se destacan en **negrita**.

4.3.1. Clasificación binaria

La clasificación binaria distingue entre los cálculos de oxalato cálcico monohidrato (COM), el tipo más frecuente en el conjunto de datos, y el resto de clases agrupadas. En esta tarea, ResNet50 obtiene el mejor rendimiento de forma consistente en todas las métricas, con un AUC del 93 % frente al 90 % del XGBoost. No obstante, las diferencias entre los tres modelos son reducidas (entre 1 y 3 puntos en la mayoría de métricas) y del mismo orden que la desviación típica entre *folds*. El rendimiento de XGBoost y del modelo híbrido resulta prácticamente equivalente, salvo en el AUC, donde el modelo híbrido alcanza el 92 %.

El comportamiento de XGBoost es coherente con lo reportado en los trabajos de referencia: una combinación adecuada de descriptores de textura (LBP) y color (HSV) permite a un modelo de aprendizaje superficial aproximarse al rendimiento de las redes profundas en este dominio.

Por otro lado, el modelo híbrido no supera al ResNet50, en contra de lo que cabría esperar de un esquema de fusión. Respecto al XGBoost obtiene un AUC ligeramente mayor, lo que indica que los descriptores manuales aportan señal discriminativa que, sin embargo, no basta para alcanzar el rendimiento del ResNet50. Para examinar el efecto de introducir los descriptores manuales en la fusión, se proyecta mediante UMAP la representación interna de cada modelo (figura 4.4): los descriptores LBP+HSV de 40 dimensiones que recibe XGBoost, y las representaciones aprendidas de 256 dimensiones de ResNet50 y del modelo híbrido, extraídas como entrada a la capa lineal final de clasificación.

Dentro de cada representación, los descriptores manuales (LBP+HSV) muestran el solapamiento más acusado: prácticamente todas las muestras de COM se confunden con las de la clase agrupada. ResNet50 ofrece una separación algo mayor, con una tendencia de cada clase a ocupar regiones distintas, aunque persiste una zona central de solapamiento. El modelo híbrido presenta un grado de separación comparable al de la red profunda, lo que resulta en un rendimiento similar entre ambos modelos.

En conjunto, más que una superioridad de un modelo sobre otro, los resultados apuntan a que la tarea binaria está próxima a la saturación: los tres modelos se sitúan en un rango de rendimiento alto y estrecho (*F1* entre el 84 % y el 86 %, AUC entre el 90 % y el 93 %), con diferencias del orden de la desviación entre *folds*. En este escenario, la mayor parte de la señal necesaria para separar COM del resto ya es capturada por cualquiera de las ramas por separado, por lo que la fusión no dispone de margen para aportar una mejora apreciable. Esto explica que el modelo híbrido no mejore al ResNet50 pese a incorporar información adicional: dicha información es en buena medida redundante con la que la red profunda ya extrae, y no una fuente de mejora en esta tarea concreta.

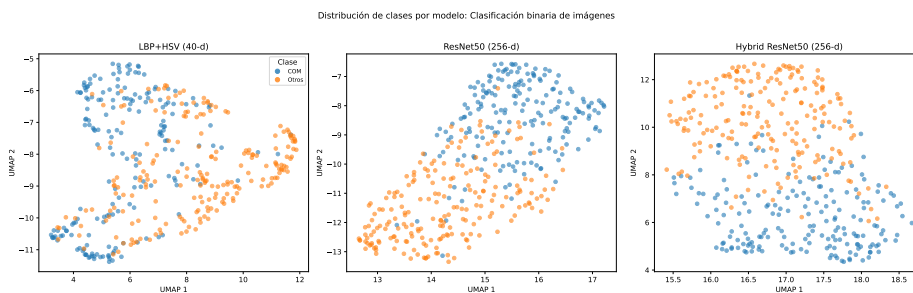


Figura 4.4: Proyección UMAP 2D de la capa de clasificación de los tres modelos en el escenario de clasificación binaria de imágenes.

4.3.2. Clasificación multiclase

En este escenario, los modelos basados en CNN emplean *weighted_cross_entropy* para compensar el desbalance entre clases. Bajo esta configuración, el patrón cambia respecto al caso binario. Por un lado, ResNet50 y XGBoost obtienen valores globales

4. RESULTADOS Y DISCUSIÓN

similares en exactitud, *F1* y exhaustividad, si bien ResNet50 supera a XGBoost en precisión (76 % frente a 71 %) y en AUC (90 % frente a 88 %). Por otro lado, el modelo híbrido alcanza el mejor *F1* global, con un 74 % frente al 70 % de ResNet50 y al 69 % de XGBoost. A diferencia del caso binario, donde la fusión no disponía de margen de mejora, aquí los descriptores manuales sí aportan información complementaria, como se detalla a lo largo de esta sección.

El análisis se apoya en tres elementos, cada uno con una función distinta: la proyección UMAP (figura 4.5) informa sobre la separabilidad de las representaciones aprendidas; las matrices de confusión (figura 4.6) muestran la dirección de los errores; y las métricas por clase (tabla 4.2) cuantifican el coste de clasificación de cada clase.

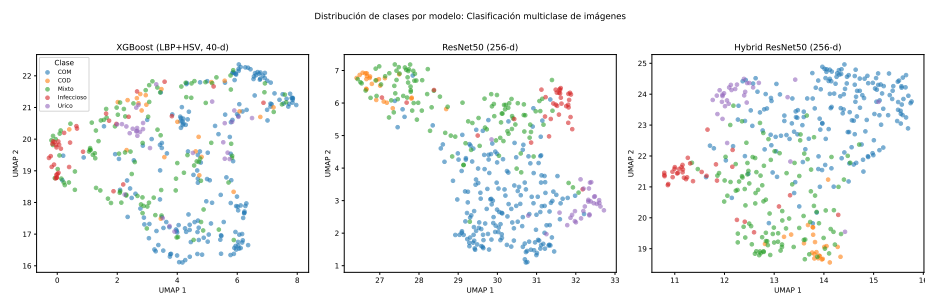


Figura 4.5: Proyección UMAP 2D de la capa de clasificación de los tres modelos en el escenario de clasificación multiclase de imágenes.

Dentro de esta lectura, los tres espacios comparten un mismo núcleo de solapamiento entre COM, Mixto y COD, mientras que Infeccioso y Úrico tienden a formar agrupamientos más definidos. Este solapamiento tiene una base composicional: la clase Mixto agrupa el subtipo 3bII (COD con contenido de HAP) junto con los subtipos mixtos COD/HAP, mientras que COD contiene el subtipo 3aII (sección 3.2) considerado COD puro. Dado que 3bII y 3aII comparten la misma base de oxalato cálcico dihidratado, la frontera entre las clases de COD y la fracción oxálica de Mixto es en parte ambigua, de modo que su solapamiento refleja una proximidad química real y no únicamente una limitación del clasificador.

Las representaciones difieren, no obstante, según el modelo. El espacio de XGBoost presenta el mayor solapamiento, sin agrupamientos definidos, lo que indica que los descriptores de textura y color (LBP+HSV) generan patrones parecidos para varias clases. ResNet50 exhibe una organización más estructurada, con regiones diferenciadas para COM, Úrico e Infeccioso, aunque COD permanece embebido en la nube de Mixto. El modelo híbrido mantiene esta estructura y define Infeccioso de forma algo más compacta, lo que sugiere que los descriptores manuales aportan una señal cromática estable para esta clase; el solapamiento entre COD y Mixto, en cambio, persiste. Esta progresión en la estructura de los espacios es coherente con el orden de rendimiento de los tres modelos.

Las matrices de confusión permiten cuantificar estos errores, y el patrón que emerge con mayor claridad es la confusión entre COD y Mixto, que se manifiesta con distinta intensidad según el modelo: mientras que en XGBoost más de la mitad de las muestras de COD (13 de 24) acaban predichas como Mixto, en las redes profundas esta fuga se reduce a aproximadamente un tercio (8 de 24). A ello se añade que el comportamiento

4.3. Clasificación usando imágenes

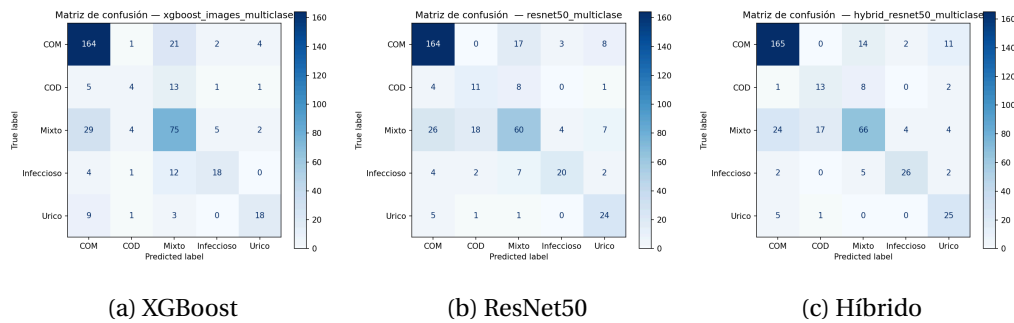


Figura 4.6: Matrices de confusión, agregadas sobre los *folds*, de la clasificación multiclase sobre imágenes completas.

de XGBoost con COD es marcadamente conservador, pues asigna la etiqueta COD en apenas 11 ocasiones en todo el conjunto, frente a las cerca de 31 de las redes profundas. Este contraste resulta coherente con el uso de la *weighted_cross_entropy*, que al incrementar el peso de las clases minoritarias en la pérdida de las CNN las empuja a predecir COD con mayor frecuencia; el resultado es una exhaustividad de COD sensiblemente mayor, si bien se obtiene a costa de arrastrar muestras de Mixto hacia COD, como refleja el hecho de que entre 17 y 18 muestras de Mixto se predicen como COD en las redes, frente a solo 4 en XGBoost.

En cuanto a Mixto, su error principal se dirige hacia COM en los tres modelos (entre 24 y 29 muestras), lo que confirma que la frontera entre las tres clases oxálicas es difusa, mientras que COM, pese a ser la clase más reconocible, cede solo una fracción reducida de sus muestras a Mixto (entre 14 y 21 de 192). Los errores de Úrico, por su parte, se concentran en COM (entre 5 y 9 muestras) sin apenas confusión con las demás clases, y no se observa prácticamente confusión mutua entre Úrico e Infeccioso en ninguno de los modelos, de modo que ambos tipos quedan bien delimitados entre sí. Por último, la fuga de Infeccioso hacia Mixto disminuye de forma progresiva a lo largo de los tres modelos (12, 7 y 5 muestras en XGBoost, ResNet50 e híbrido, respectivamente), una tendencia que concuerda con la mayor compacidad de Infeccioso observada en la proyección del modelo híbrido.

Clase	XGBoost			ResNet50			Híbrido			Soporte
	Precisión	Exhaustividad	<i>F1</i>	Precisión	Exhaustividad	<i>F1</i>	Precisión	Exhaustividad	<i>F1</i>	
COM	78 ± 6	85 ± 8	81 ± 5	85 ± 8	85 ± 10	84 ± 4	87 ± 8	84 ± 11	85 ± 6	192
COD	47 ± 32	17 ± 11	22 ± 12	40 ± 18	57 ± 25	42 ± 10	41 ± 15	53 ± 29	42 ± 16	24
Mixto	61 ± 7	65 ± 2	63 ± 3	63 ± 4	52 ± 15	56 ± 9	66 ± 9	56 ± 13	59 ± 7	115
Infeccioso	74 ± 22	51 ± 8	58 ± 11	81 ± 24	71 ± 16	70 ± 6	75 ± 24	73 ± 15	70 ± 10	35
Úrico	79 ± 19	64 ± 23	69 ± 18	61 ± 11	79 ± 21	67 ± 13	58 ± 10	80 ± 16	67 ± 12	31

Tabla 4.2: Resultados por clase (Precisión, Exhaustividad y *F1*, en %) para los tres modelos bajo la misma estrategia de validación. Los valores se expresan como media ± desviación estándar, en porcentaje; la columna Soporte indica el número de muestras por clase.

Finalmente, las métricas por clase permiten cuantificar el coste de clasificación asociado a cada una de ellas, y de su lectura se desprende que COM, la clase mayoritaria con 192 muestras, se mantiene como la más reconocible, con un *F1* superior al 81 % en los tres modelos, mientras que COD se sitúa en el extremo opuesto como la más difícil,

con un *F1* que oscila entre el 22 % del XGBoost y el 42 % de las redes profundas y con desviaciones muy elevadas que alcanzan hasta el 32 % en la precisión del XGBoost.

Un patrón distinto aparece en Mixto, cuya situación resulta reveladora, pues, pese a contar con 115 muestras —un soporte muy superior al de Infeccioso (35) y Úrico (31)—, su *F1* (entre el 56 y el 63 %) es comparable o incluso inferior al de ambas, lo que indica que su dificultad no proviene de la falta de muestras, sino de su heterogeneidad interna, en coherencia con el solapamiento difuso observado en la proyección UMAP. Conviene señalar, además, que Mixto es la única clase en la que XGBoost (*F1* del 63 %) supera a las redes profundas (56–59 %), lo que sugiere que los descriptores de textura y color capturan para esta clase una señal útil que las representaciones profundas no llegan a recuperar por completo.

Este mismo efecto explica dónde aporta valor la fusión, ya que la mejora del modelo híbrido respecto a ResNet50 se concentra precisamente en Mixto (+3 puntos de *F1*) y, en menor medida, en COM (+1 punto), mientras que las clases minoritarias mantienen prácticamente el mismo *F1*; en otras palabras, los descriptores LBP+HSV aportan información complementaria en aquellas clases que combinan un soporte suficiente con una morfología confundible, pero no facilitan la detección de las clases con pocas muestras.

Asimismo, las redes profundas detectan las clases minoritarias con mayor exhaustividad que XGBoost, aunque de forma menos conservadora, como se aprecia en COD, donde XGBoost obtiene una exhaustividad del 17 % frente al 53–57 % de las redes, y en Úrico, donde XGBoost prioriza la precisión (79 %) sobre la exhaustividad (64 %) mientras que las CNN invierten el patrón, con una exhaustividad del 79–80 % y una precisión del 58–61 %. Esta inversión entre precisión y exhaustividad es consistente con la ponderación de la pérdida de las redes profundas, que penaliza en mayor medida los fallos sobre las clases minoritarias y desplaza su umbral efectivo hacia una mayor cobertura.

En conjunto, los tres análisis convergen en una misma conclusión: la limitación dominante reside en la separabilidad del núcleo COM–COD–Mixto, cuya frontera es difusa por la proximidad química entre los subtipos oxálicos, y no en la capacidad de los modelos. Sobre esta limitación de fondo, COD arrastra un segundo problema de naturaleza estadística: su escaso soporte (24 muestras, 4 o 5 por *fold*) hace que sus métricas sean muy inestables, hasta el punto de que un único acierto o error desplaza notablemente el resultado. Ambos factores se distinguen por su efecto: un mayor número de muestras reduciría con seguridad esta varianza y haría las estimaciones más fiables, si bien solo elevaría el rendimiento en la medida en que las nuevas muestras aportaran señal discriminativa adicional, algo que la proximidad composicional con Mixto no garantiza. Infeccioso y Úrico, en cambio, químicamente más distintas, alcanzan un rendimiento superior pese a su bajo número de muestras, lo que confirma que el soporte reducido, por sí solo, no es el factor determinante.

Técnicas de balanceo aplicadas a XGBoost

En este apartado se evalúan tres técnicas para mitigar el efecto del desbalance en el XGBoost: la ponderación de muestras (*sample_weight*), el sobremuestreo sintético (SMOTE) y la aumentación de imágenes.

4.3. Clasificación usando imágenes

Modelo	Exactitud	<i>F1</i>	Precisión	Exhaustividad	AUC
<i>Baseline</i>	70 ± 4	69 ± 4	71 ± 4	70 ± 4	88 ± 4
<i>Sample Weight</i>	67 ± 7	67 ± 6	68 ± 6	67 ± 7	88 ± 4
Aumentación	69 ± 4	68 ± 3	69 ± 2	69 ± 4	87 ± 4
SMOTE	68 ± 5	68 ± 5	70 ± 4	68 ± 5	87 ± 4

Tabla 4.3: Resultados de las técnicas de balanceo aplicadas a XGBoost para la clasificación multiclase de imágenes.

La tabla 4.3 muestra que ninguna de las tres técnicas logra mejorar el *baseline*, sino que todas producen un ligero descenso en el rendimiento global, siendo la ponderación de muestras la que peor resultado ofrece y, a la vez, la que presenta mayor varianza entre *folds*. Esa mecánica de ponderación balanceada aplicada asigna a cada muestra un peso inversamente proporcional a la frecuencia de su clase, de modo que las muestras de las clases minoritarias contribuyen con un peso muy superior al de la mayoritaria a la pérdida que XGBoost minimiza durante el entrenamiento. Como resultado, esa pérdida queda dominada por un número reducido de muestras minoritarias, y el ajuste del modelo se orienta a acomodarlas aun cuando estén dispersas o solapadas; los árboles pasan así a depender en gran medida de qué muestras concretas resultan influyentes en cada *fold*, lo que se traduce en un sobreajuste a esas muestras y en la elevada varianza observada entre particiones.

Clase	Baseline			Baseline+SMOTE			Soporte
	Precisión	Exhaustividad	<i>F1</i>	Precisión	Exhaustividad	<i>F1</i>	
COM	78 ± 6	85 ± 8	81 ± 5	81 ± 4	81 ± 8	81 ± 5	192
COD	47 ± 32	17 ± 11	22 ± 12	20 ± 21	23 ± 20	21 ± 20	24
Mixto	61 ± 7	65 ± 2	63 ± 3	64 ± 7	59 ± 13	60 ± 7	115
Infeccioso	74 ± 22	51 ± 8	58 ± 11	59 ± 20	58 ± 11	56 ± 11	35
Úrico	79 ± 19	64 ± 23	69 ± 18	66 ± 15	70 ± 25	67 ± 17	31

Tabla 4.4: Resultados detallados por clase (Precisión, Exhaustividad y *F1*, en %) para el modelo XGBoost comparando la estrategia Baseline frente a Baseline+SMOTE. Los valores se expresan como media ± desviación estándar, en porcentaje. La columna Soporte indica el número de muestras por clase.

Para comprender el efecto del SMOTE se analizan las métricas por clase (tabla 4.4) comparando el modelo del *Baseline* con el que incorpora esta técnica, la cual genera muestras sintéticas de las clases minoritarias interpolando en el espacio de características LBP+HSV hasta igualar su número al de la clase mayoritaria dentro del conjunto de entrenamiento de cada *fold*. Dado el fuerte desbalance del conjunto, esto implica una proporción muy elevada de muestras sintéticas, hasta el punto de que, para COD, se generan del orden de ocho muestras interpoladas por cada muestra real.

El efecto general del SMOTE consiste en incrementar la exhaustividad de las clases minoritarias a costa de su precisión, es decir, el modelo predice estas clases con más frecuencia pero también genera más falsos positivos, de manera que la exhaustividad de las clases Infeccioso y Úrico sube en 6–7 puntos mientras que su precisión desciende de forma más acusada, hasta 16 puntos. Este reequilibrio reduce el *F1* en la mayoría de

clases —Mixto del 63 % al 60 %, Infeccioso del 58 % al 56 % y Úrico del 69 % al 67 %—, con la excepción de COM, que apenas se ve afectada ($F1$ del 81 % en ambos casos), ya que su reequilibrio interno entre precisión y exhaustividad deja el rendimiento prácticamente intacto.

El caso más revelador es el de COD, donde la precisión cae del 47 % al 20 %, lo que indica que el modelo predice esta clase con mucha más frecuencia pero de forma mayoritariamente incorrecta; a ello se suma que sus métricas bajo SMOTE presentan desviaciones muy elevadas entre *folds* (precisión de 20 ± 21 y $F1$ de 21 ± 20), lo que refleja que la clase no llega a alcanzar un comportamiento estable, pues al interpolar entre las escasas muestras reales de COD en algunos *folds* se generan muestras sintéticas útiles y en otros muestras que caen en regiones ambiguas del espacio, ya solapadas con Mixto. En última instancia, la interpolación no crea información nueva, sino que se limita a densificar una región donde las clases oxálicas ya se confunden de por sí.

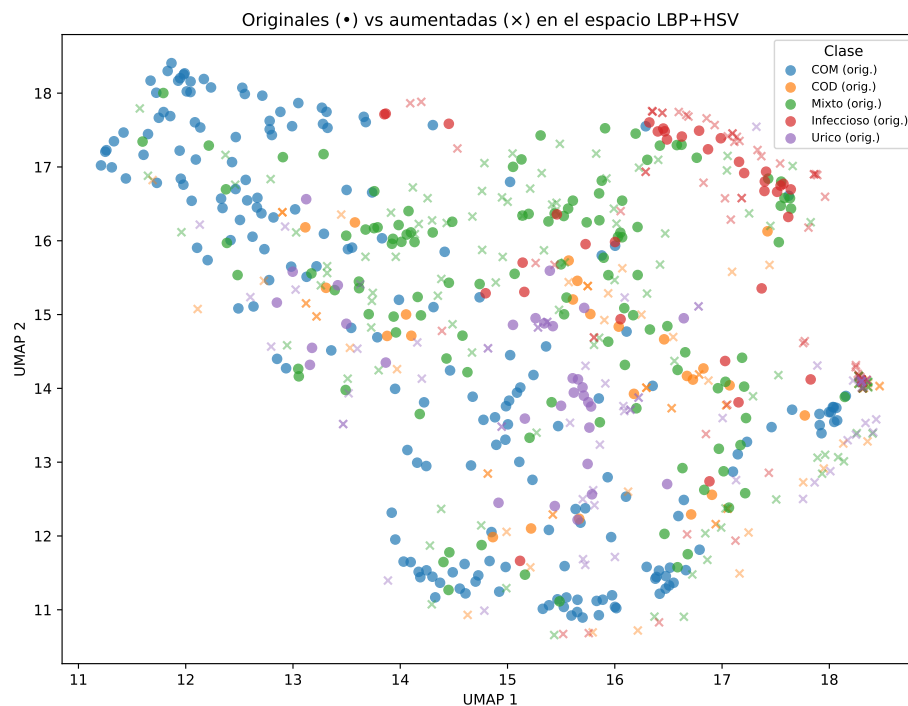


Figura 4.7: Proyección del espacio de características LBP+HSV de las imágenes originales (puntos) y sus versiones aumentadas (cruces), coloreadas por clase.

Finalmente, la aumentación de imágenes también degrada el rendimiento respecto al *baseline*. Las transformaciones aplicadas para generar las nuevas imágenes combinan volteos horizontal y vertical, ajustes de brillo y contraste, variaciones leves de tono, saturación y valor, ecualización adaptativa del histograma (CLAHE) y ruido gaussiano, evitando alteraciones cromáticas fuertes que pudieran modificar los matices que distinguen unos cálculos de otros.

La proyección de la figura 4.7 revela que las muestras aumentadas se distribuyen aproximadamente sobre la región que ocupan las originales de su misma clase, sin desplazarse de forma masiva hacia zonas ajenas, de manera que, al caer sobre esa misma región del espacio, se limitan a reforzar la densidad en áreas que ya presentan

solapamiento entre clases sin aportar capacidad discriminativa adicional; en consecuencia, la aumentación no genera información útil para separar mejor las clases y termina degradando ligeramente el rendimiento.

En conclusión, ninguna de las tres técnicas evaluadas mejora el rendimiento del XGBoost, por lo que se conserva la configuración *baseline* para este modelo, un resultado que se considera coherente con la hipótesis central del trabajo, ya que el comportamiento observado —el SMOTE y la ponderación incrementan la cobertura de las clases minoritarias a costa de degradar su precisión, mientras que la aumentación no aporta información discriminativa nueva— indica que la limitación principal no reside en el desbalance, sino en la separabilidad de las clases.

En el caso de las redes profundas, el desbalance se aborda mediante la *weighted_cross_entropy* en el escenario multiclase, una función que penaliza en mayor medida los errores cometidos sobre las clases minoritarias y que, a diferencia de la ponderación empleada en XGBoost, actúa sobre representaciones aprendidas de mayor capacidad discriminativa y en un régimen de entrenamiento —descenso de gradiente estocástico por lotes sobre una red preentrenada— menos sensible a la inestabilidad. La aumentación de datos, por su parte, se emplea de forma sistemática durante el entrenamiento de las redes, si bien no como técnica de balanceo, sino como mecanismo de regularización orientado a mitigar el sobreajuste.

4.4. Clasificación usando parches

Esta sección presenta los resultados de los clasificadores cuando la entrada son parches extraídos de las imágenes, con el objetivo de determinar si su uso ofrece alguna ventaja sobre la clasificación directa de imágenes. Para ello se reportan las métricas a nivel de parche y, mediante agregación, a nivel de imagen, lo que permite una comparación directa con los resultados de la sección anterior.

4.4.1. Clasificación binaria

A nivel de parche, el rendimiento de los modelos es inferior al obtenido sobre imágenes completas, ya que el *F1* desciende del rango 84–86 % sobre imágenes al 80–81 % sobre parches, una caída que se explica por la pérdida de contexto que implica trabajar sobre una región acotada del cálculo: al disponer de menos información en cada decisión, el modelo prescinde del contexto global que sí ofrece la imagen completa.

Modelo	Clasificación binaria a nivel de parche					Clasificación binaria agregada por imagen				
	Exactitud	<i>F1</i>	Precisión	Exhaustividad	AUC	Exactitud	<i>F1</i>	Precisión	Exhaustividad	AUC
XGBoost	80 ± 2	80 ± 2	81 ± 2	80 ± 2	88 ± 1	84 ± 5	84 ± 5	84 ± 5	84 ± 5	92 ± 3
ResNet50	81 ± 3	81 ± 3	81 ± 3	81 ± 3	90 ± 2	86 ± 4	86 ± 4	88 ± 2	86 ± 4	93 ± 3
Híbrido	81 ± 3	81 ± 3	82 ± 3	81 ± 3	90 ± 1	86 ± 5	85 ± 5	88 ± 2	86 ± 5	94 ± 3

Tabla 4.5: Resultados de clasificación binaria (COM vs. Otros) sobre parches, a nivel de parche y agregados por imagen. Los mejores resultados de cada bloque se destacan en **negrita**.

En la comparación entre modelos, las redes profundas mejoran ligeramente al XGBoost, como cabía esperar por su mayor capacidad de representación, mientras que

el modelo híbrido rinde de forma equivalente al ResNet50 sobre parches, sin diferencia apreciable entre ambos.

Comparación entre parches e imágenes completas

Para determinar si el enfoque por parches resulta más efectivo que el de imágenes completas, ambas estrategias deben compararse sobre el mismo nivel de agregación, por lo que las predicciones a nivel de parche se combinan a nivel de imagen mediante *soft voting* —el promedio de las probabilidades predichas para todos los parches de una misma imagen—, obteniendo así una única decisión por imagen comparable con la de la clasificación directa.

Esta agregación eleva las métricas respecto al nivel de parche de forma consistente en los tres modelos, como ilustra el caso del modelo híbrido, cuyo *F1* pasa del 81 % al 85 % y cuyo AUC sube del 90 % al 94 %. Este incremento, no obstante, no debe interpretarse como una ventaja del enfoque por parches, sino como la recuperación del rendimiento perdido al fragmentar la imagen, ya que el promedio de las predicciones de los parches actúa como un *ensemble* que compensa la falta de contexto de cada parche individual. De hecho, al situar la comparación en el nivel de agregación relevante —imágenes completas frente a parches agregados por imagen—, los resultados son prácticamente idénticos: el *F1* sobre imágenes completas (84, 86 y 84 % para XGBoost, ResNet50 e híbrido) coincide con el obtenido tras agregar los parches (84, 86 y 85 %), y las diferencias en AUC son de menor magnitud. En la tarea binaria, por tanto, el enfoque por parches no aporta ninguna ventaja sobre la clasificación directa de la imagen completa, puesto que la fragmentación seguida de agregación se limita a recuperar el rendimiento de partida sin llegar a superarlo.

4.4.2. Clasificación multiclase

Antes de presentar los resultados de la clasificación multiclase sobre parches, se compara el rendimiento de los modelos con y sin aumentación de parches, para justificar la configuración empleada en el resto de la evaluación.

Modelo	Baseline (sin aumentación)					Baseline + aumentación de parches				
	Exactitud	<i>F1</i>	Precisión	Exhaustividad	AUC	Exactitud	<i>F1</i>	Precisión	Exhaustividad	AUC
XGBoost	66 ± 4	65 ± 3	68 ± 2	66 ± 4	85 ± 3	55 ± 4	55 ± 5	58 ± 4	55 ± 4	83 ± 2
ResNet50	69 ± 2	67 ± 2	68 ± 2	69 ± 2	87 ± 1	62 ± 5	62 ± 5	64 ± 3	62 ± 5	88 ± 3
Híbrido	69 ± 2	68 ± 2	70 ± 2	69 ± 2	88 ± 1	63 ± 5	63 ± 5	65 ± 4	63 ± 5	89 ± 3

Tabla 4.6: Comparación de las métricas de los modelos con y sin aumentación de parches en la clasificación multiclase a nivel de parche. Los mejores resultados de cada tarea se destacan en **negrita**.

La tabla 4.6 muestra que la aumentación de parches empeora sistemáticamente el rendimiento de los tres modelos, con caídas notables en exactitud y *F1*, en contra de lo reportado por López *et al.* [4], donde la extracción de parches adicionales mejora el rendimiento. XGBoost es el más perjudicado, con un descenso de 11 puntos en exactitud (del 66 % al 55 %) y de 10 en *F1*, mientras que las redes profundas caen en torno a 5 puntos de *F1* (ResNet50 del 67 % al 62 % e híbrido del 68 % al 63 %), una diferencia coherente con la naturaleza de cada modelo, puesto que el XGBoost opera

exclusivamente sobre descriptores LBP+HSV y las transformaciones aplicadas alteran directamente su espacio de entrada, ya de por sí poco separable, mientras que las redes profundas procesan los píxeles y toleran mejor las variaciones fotométricas leves, por lo que su degradación es menor aunque tampoco en su caso la aumentación llegue a aportar mejora.

Resulta llamativo que, pese a la caída en exactitud y $F1$, el AUC se mantenga relativamente estable, con un descenso de apenas dos puntos en el XGBoost y sin cambios apreciables en las redes profundas. Dado que el AUC mide la capacidad de ordenamiento de las clases mientras que la exactitud y el $F1$ dependen de la asignación final por *argmax*, esta divergencia indica que la aumentación no destruye la estructura discriminativa aprendida, sino que degrada la frontera de decisión y multiplica los errores de asignación, un comportamiento que se explica por la propia estructura del espacio de características: mientras que en el trabajo de López *et al.* [4] las clases presentan una separabilidad suficiente para que los parches adicionales refuercen agrupamientos ya definidos, en este conjunto las clases se solapan de forma considerable, de modo que los parches aumentados solo incrementan la densidad de muestras en un espacio ya disperso.

Por tanto, la aumentación de parches reproduce lo ya observado con las técnicas de balanceo sobre imágenes (sección 4.3.2): no mejora la separabilidad entre clases, que sigue siendo la limitación de fondo, razón por la cual el resto de la evaluación multiclase sobre parches se realiza sin aumentación.

Modelo	Clasificación multiclase a nivel de parche					Clasificación multiclase agregada por imagen				
	Exactitud	$F1$	Precisión	Exhaustividad	AUC	Exactitud	$F1$	Precisión	Exhaustividad	AUC
XGBoost	66 ± 4	65 ± 3	68 ± 2	66 ± 4	85 ± 3	71 ± 5	69 ± 5	70 ± 6	71 ± 5	89 ± 4
ResNet50	69 ± 2	67 ± 2	68 ± 2	69 ± 2	87 ± 1	72 ± 4	69 ± 5	70 ± 5	72 ± 4	91 ± 2
Híbrido	69 ± 2	68 ± 2	70 ± 2	69 ± 2	88 ± 1	74 ± 5	71 ± 6	73 ± 6	74 ± 5	92 ± 2

Tabla 4.7: Resultados de la clasificación multiclase sobre parches, a nivel de parche y agregados por imagen. La columna a nivel de parche corresponde a la configuración sin aumentación de la tabla 4.6. Los mejores resultados de cada tarea se destacan en **negrita**.

Clase	XGBoost			ResNet50			Híbrido			Soporte
	Precisión	Exhaustividad	$F1$	Precisión	Exhaustividad	$F1$	Precisión	Exhaustividad	$F1$	
COM	75 ± 3	84 ± 6	79 ± 4	76 ± 3	89 ± 4	82 ± 2	76 ± 4	89 ± 4	82 ± 3	2767
COD	45 ± 17	31 ± 25	30 ± 8	40 ± 15	22 ± 8	25 ± 4	44 ± 11	20 ± 5	27 ± 5	298
Mixto	51 ± 15	52 ± 2	51 ± 9	54 ± 8	51 ± 5	52 ± 4	55 ± 9	53 ± 6	53 ± 3	1372
Infeccioso	60 ± 19	50 ± 15	50 ± 8	62 ± 18	53 ± 13	54 ± 8	59 ± 21	54 ± 15	51 ± 6	485
Úrico	60 ± 14	57 ± 19	55 ± 11	65 ± 12	61 ± 14	61 ± 10	74 ± 9	61 ± 16	65 ± 10	368

Tabla 4.8: Resultados detallados por clase (Precisión, Exhaustividad y $F1$, en %) de los tres clasificadores evaluados en el escenario de clasificación multiclase de parches. Los valores se expresan como media ± desviación estándar, en porcentaje. La columna Soporte indica el número de muestras por clase.

En este escenario, el modelo híbrido obtiene los mejores valores globales en la mayoría de métricas, si bien el análisis por clase revela que esta ventaja no es transversal, sino que se concentra en clases concretas. La diferencia más nítida se da en Úrico, donde el híbrido eleva la precisión del 65 % al 74 % manteniendo la exhaustividad (61 %), lo que se traduce en una reducción de los falsos positivos: detecta los mismos

4. RESULTADOS Y DISCUSIÓN

casos que el ResNet50 pero deja de asignar a esta clase muestras que no le corresponden. En COD y Mixto las diferencias son menores y se consideran equivalentes —en COD, la ligera ganancia en *F1* responde a un intercambio entre precisión y exhaustividad, no a un mayor número de aciertos—, mientras que en Infeccioso el híbrido no solo no mejora, sino que reduce su *F1* del 54 % al 51 %, en contraste con el escenario de imágenes, donde ambos modelos igualaban su rendimiento en esta clase.

Este patrón indica que la incorporación de los descriptores LBP+HSV solo aporta una mejora apreciable en aquellas clases cuya señal de color o textura captura bien la rama manual, sin alterar el rendimiento en el núcleo COM–COD–Mixto, que continúa siendo la principal fuente de confusión. A continuación se interpretan los espacios de características de cada modelo en este escenario (figura 4.8).

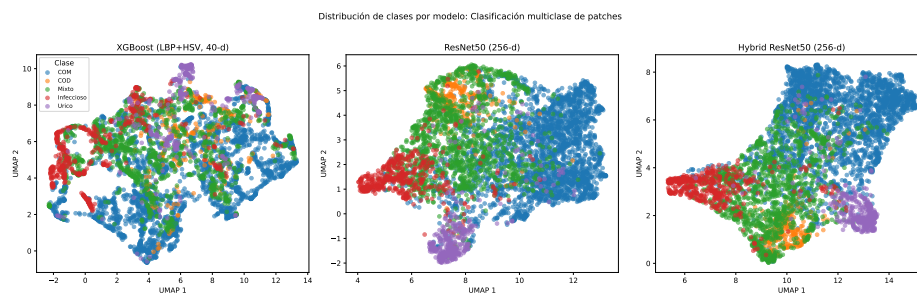


Figura 4.8: Proyección UMAP 2D de la capa de clasificación de los tres modelos en el escenario de clasificación multiclase de parches.

Dado el elevado número de parches extraídos por imagen, el espacio de características es más denso que en el escenario de imágenes completas, y en él el espacio LBP+HSV vuelve a mostrarse como el más disperso, con un fuerte solapamiento entre clases y sin que ninguna llegue a formar un agrupamiento compacto: COM y Mixto ocupan varias regiones del mapa y se solapan entre sí, mientras que Infeccioso forma un pequeño agrupamiento del que, no obstante, varias muestras se dispersan hacia regiones de COM, COD, Mixto e incluso Úrico. Esta representación poco estructurada es coherente con el rendimiento moderado del XGBoost.

ResNet50 e híbrido proporcionan representaciones más compactas, en las que cuatro de las cinco clases (todas excepto COD) ocupan una región propia y forman un agrupamiento identificable, si bien persiste solapamiento en los bordes que genera una zona central ambigua donde conviven muestras de varias categorías; COD, en cambio, se solapa fuertemente con Mixto, hasta el punto de poder interpretarse casi como una subregión de esta. Entre ambas redes, las proyecciones son muy similares y no se aprecia visualmente una mejora clara en la del modelo híbrido, de modo que las diferencias de rendimiento observadas entre ambos —favorables al híbrido en clases como Úrico— no tienen un reflejo evidente en la estructura bidimensional.

4.4.3. Imágenes completas frente a parches

Una vez evaluados ambos tipos de entrada, es posible comparar directamente su eficacia sobre el mismo nivel de agregación. La tabla 4.9 confronta las métricas obtenidas sobre imágenes con las de los parches agregados por imagen, en las dos tareas.

Modelo	Binaria (<i>F1</i>)		Multiclase (<i>F1</i>)	
	Imagen	Parche agregado	Imagen	Parche agregado
XGBoost	84 ± 4	84 ± 5	69 ± 4	69 ± 5
ResNet50	86 ± 2	86 ± 4	70 ± 4	69 ± 5
Híbrido	84 ± 3	85 ± 5	74 ± 3	71 ± 6

Tabla 4.9: Comparación del *F1* entre la clasificación sobre imágenes completas y la clasificación sobre parches agregada por imagen, para ambas tareas.

En la tarea binaria, ambos enfoques alcanzan métricas equivalentes: las diferencias en *F1* no superan un punto y quedan dentro del rango de la desviación entre *folds*. En la tarea multiclase, las imágenes igualan o superan a los parches en los tres modelos; en particular, el modelo híbrido —el de mejor rendimiento global— obtiene un *F1* del 74 % sobre imágenes frente al 71 % sobre parches agregados.

Por tanto, en ninguna de las dos tareas el enfoque por parches aporta una ventaja de rendimiento sobre la clasificación directa de imágenes: como ya se detalló para el caso binario, la agregación recupera el rendimiento que se pierde al fragmentar la imagen, pero no llega a superar el que se obtiene procesándola completa.

4.5. Comparación con el estado del arte

La comparación directa de los resultados obtenidos con los reportados por el grupo de López *et al.* [4] resulta limitada por una diferencia taxonómica: el conjunto de datos empleado en este trabajo se clasifica según la taxonomía de Grases *et al.* [3], mientras que dicho grupo emplea la de Daudon *et al.* [11], y no existe, aún, una correspondencia oficial y completa entre ambas.

Concretamente, el trabajo de referencia clasifica los cuatro tipos de cálculos más frecuentes: COM, COD, UA y BRU. El presente trabajo, en cambio, emplea cinco categorías: COM, COD, mixto, infeccioso y úrico. La tabla 4.10 refleja la correspondencia posible entre las clases de ambos trabajos.

Como se observa, la correspondencia es parcial: algunas clases de Daudon engloban una o más de Grases. Por ello la comparación de resultados es imprecisa, ya que no hay una relación directa y completa entre ambas clasificaciones. Por consiguiente, la comparación que sigue es a escala general, contrastando el rendimiento de los modelos en función de sus clases equivalentes.

Las tablas 4.11 y 4.8 recogen, respectivamente, los resultados de López *et al.* [4] y los obtenidos en el presente TFG, ambos promediados de forma ponderada por el soporte de cada clase, lo que los hace comparables en ese aspecto. La comparación se limita, no obstante, a las clases con equivalente en los dos trabajos, puesto que las categorías mixta e infecciosa del presente trabajo carecen de contrapartida en el conjunto de López *et al.* [4], que no incluye cálculos de composición mixta ni infecciosa. Como cabía esperar, los resultados de este trabajo son inferiores a los del estado del arte en todas las clases evaluadas, una diferencia que responde principalmente a dos factores: el diseño de las clases y el protocolo de evaluación.

4. RESULTADOS Y DISCUSIÓN

Daudon	Grases	Correspondencia
Ia	1aI 2a	100 % 50–70 % ¹
IIb	3aII 3bII	100 % ≈ 50 % ^{1,2}
IIIb	8	Parcial ³
IVd	7	100 % ⁴

¹ Estimación aproximada.

² El porcentaje depende del contenido en HAP.

³ Los cálculos de ácido úrico clasificados como tipo 8 por Grases no corresponden todos al subtipo IIIb de Daudon: según su morfología, pueden corresponder también a los subtipos IIIa o IIIc.

⁴ Grupo no incluido en la selección de clases empleada en este trabajo.

Tabla 4.10: Correspondencia aproximada entre los tipos de la clasificación de Daudon y los de la clasificación de Grases.

Clase	XGBoost			InceptionV3		
	Precisión	Exhaustividad	<i>F1</i>	Precisión	Exhaustividad	<i>F1</i>
COM (Ia)	92 ± 18	96 ± 18	93 ± 18	98 ± 06	97 ± 05	97 ± 06
COD (IIb)	91 ± 18	91 ± 18	91 ± 19	93 ± 09	96 ± 09	94 ± 09
UA (IIIb)	97 ± 10	96 ± 19	96 ± 19	95 ± 08	90 ± 08	90 ± 08
BRU (IVd)	96 ± 19	94 ± 19	92 ± 19	96 ± 08	95 ± 09	95 ± 08

Tabla 4.11: Resultados por clase (Precisión, Exhaustividad y *F1*, en%) para los modelos evaluados en el trabajo de referencia de López *et al.* [4]. Los valores de *F1* se han calculado a partir de la precisión y la exhaustividad reportadas en su tabla de resultados por clase.

Por un lado, las clases de este trabajo son más difíciles de separar por construcción, ya que, mientras que los cuatro tipos del trabajo de referencia están formados por subtipos puros (Ia, IIb, IIIb, IVd) —hasta el punto de que sus autores excluyen los cálculos mixtos y los proponen como línea de trabajo futuro—, aquí varias clases agrupan bajo una misma etiqueta subtipos morfológicamente distintos: COM combina 1aI y 2a, la clase mixta reúne 3bII y 4I–4II, y la úrica engloba 8I y 8II. La mayor varianza intraclase no es, por tanto, una propiedad del conjunto de datos, sino una consecuencia directa de este agrupamiento, de modo que el presente trabajo aborda deliberadamente el problema que el de referencia deja fuera.

Finalmente, la evaluación es más estricta, pues se emplea *StratifiedGroupKFold* agrupado por *stone_id*, lo que impide que dos parches de un mismo cálculo se repartan entre entrenamiento y prueba; el trabajo de referencia, en cambio, no describe una agrupación equivalente al construir sus particiones, de manera que, si estas se generan a nivel de parche, parches procedentes de un mismo cálculo podrían aparecer simultáneamente en ambos conjuntos e introducir un sesgo optimista en las métricas. Por consiguiente, las cifras de ambos trabajos no son estrictamente comparables: las de este trabajo son más bajas, pero representativas del rendimiento sobre cálculos no vistos.

4.6. Estudio de ablación: histogramas de color frente a energías HSV

Como se justificó en la sección 3.4.1, este trabajo emplea histogramas de color HSV como descriptor cromático, en lugar de las energías del HSV que López-Tiro *et al.* [4] identifican como la mejor opción. Dado que sus resultados se obtienen sobre parches, mientras que aquí se trabaja también con imágenes, esta sección evalúa empíricamente ambas representaciones para verificar cuál resulta más adecuada en el presente contexto.

Tarea	Características	Exactitud	AUC	<i>FI</i>	Precisión	Exhaustividad
Binaria	Energías HSV	78 ± 5	84 ± 3	78 ± 5	78 ± 5	78 ± 5
	Histogramas de color	84 ± 4	90 ± 3	84 ± 4	84 ± 4	84 ± 4
Multiclase	Energías HSV	64 ± 6	79 ± 5	62 ± 5	64 ± 4	64 ± 6
	Histogramas de color	67 ± 7	88 ± 4	67 ± 6	68 ± 6	67 ± 7

Tabla 4.12: Resultados de validación cruzada (5 *folds*) comparando energías HSV e histogramas de color sobre imágenes completas, agrupados por tarea. Las métricas AUC, *FI*, precisión y exhaustividad corresponden a sus variantes ponderadas. Los valores se expresan como media ± desviación estándar, en porcentaje. Los mejores resultados de cada tarea se destacan en **negrita**.

Tarea	Características	Exactitud	AUC	<i>FI</i>	Precisión	Exhaustividad
Binaria	Energías HSV	74 ± 1	82 ± 1	74 ± 1	75 ± 1	74 ± 1
	Histogramas de color	80 ± 2	88 ± 1	80 ± 2	81 ± 2	80 ± 2
Multiclase	Energías HSV	61 ± 3	80 ± 3	59 ± 3	61 ± 2	61 ± 3
	Histogramas de color	66 ± 4	85 ± 3	65 ± 3	68 ± 2	66 ± 4

Tabla 4.13: Resultados de validación cruzada (5 *folds*) comparando energías HSV e histogramas de color con el XGBoost sobre parches, a nivel de parche, agrupados por tarea. Las métricas AUC, *FI*, precisión y exhaustividad corresponden a sus variantes ponderadas. Los valores se expresan como media ± desviación estándar, en porcentaje. Los mejores resultados de cada tarea se destacan en **negrita**.

La comparación de las tablas 4.12 y 4.13 constituye un experimento controlado: al cambiar únicamente la forma de calcular los histogramas de color —y mantener constantes el descriptor de textura (LBP), la dimensión del histograma, los datos y la estrategia de validación—, la diferencia en los resultados es atribuible exclusivamente a la representación del color. Bajo estas condiciones, los histogramas de color superan a las energías HSV de forma sistemática, con diferencias de entre 3 y 9 puntos según la métrica. La ventaja se mantiene en las cuatro condiciones evaluadas —las dos tareas y las dos escalas de entrada—, lo que indica que no responde a las particularidades de una tarea o de un nivel de agregación concretos, sino que es una propiedad de la propia representación.

Un aspecto relevante es que la mejora no se limita a las métricas de asignación (exactitud, *FI*), sino que alcanza también al AUC, cuyo incremento indica que los

histogramas de color no se limitan a mejorar la asignación final, sino que producen una estructura de salida más separable entre clases. En otras palabras, la ventaja del color se manifiesta en la propia discriminabilidad de la representación, y no únicamente en la frontera de decisión.

Ambos descriptores, pese a partir del mismo espacio HSV, codifican información distinta. El histograma de color básico cuenta cuántos píxeles del cálculo presentan cada valor de tono, saturación y valor; describe, por tanto, qué colores aparecen y en qué proporción, tratando cada píxel de forma aislada. Las energías del HSV, en cambio, se calculan a partir del gradiente de cada canal, es decir, de la diferencia entre cada píxel y sus vecinos; describen cuánto varía el color a lo largo de la superficie del cálculo, y no su valor absoluto. Dicho de otro modo, el primer descriptor conserva el color en sí, mientras que el segundo conserva su variación local y descarta el color absoluto.

En el conjunto de datos de este trabajo, la representación basada en el color absoluto resulta consistentemente más discriminante. Establecer el mecanismo exacto de esta diferencia —si el histograma de color aporta información discriminante ausente en las energías, o si estas resultan parcialmente redundantes con la textura que ya captura el LBP, también construido a partir de comparaciones entre píxeles vecinos— requeriría un análisis específico de la contribución individual de cada descriptor, que queda fuera del alcance de este trabajo y se plantea como línea de investigación futura.

CONCLUSIONES

Este TFG ha abordado la clasificación automática de cálculos renales a partir de imágenes del BICUIB, comparando tres modelos (XGBoost, ResNet50 e híbrido) sobre dos tipos de entrada (imágenes y parches) y dos tareas (binaria y multiclase). A continuación se recogen las principales conclusiones derivadas del trabajo, junto con sus limitaciones y las líneas que abren.

5.1. Rendimiento de los modelos

El comportamiento de los modelos depende en gran medida de la dificultad de la tarea. En la clasificación binaria, los tres alcanzan un rendimiento alto y similar ($F1$ entre el 84 % y el 86 %), sin que ninguno destaque de forma apreciable, ya que las diferencias observadas quedan dentro del rango de la desviación entre *folds*. En la clasificación multiclase, en cambio, el modelo híbrido obtiene el mejor rendimiento ($F1$ del 74 %), por delante del ResNet50 (70 %) y del XGBoost (69 %), lo que indica que la fusión de descriptores manuales y representaciones profundas aporta valor precisamente en la tarea más difícil, donde existe margen de mejora, y no en la binaria, próxima a la saturación.

En conjunto, las redes profundas aprenden una representación más discriminativa que la del XGBoost, lo que se traduce en un mejor rendimiento en las tareas más exigentes; no obstante, cabe destacar que el XGBoost, pese a apoyarse únicamente en descriptores manuales de color y textura, mantiene un rendimiento moderado y comparable al del ResNet50 en varios escenarios.

El análisis por clase revela, además, que la dificultad no se distribuye por igual y que sus causas difieren de una clase a otra. Un primer hallazgo, que matiza la lectura habitual del desbalance, es que el número de muestras no basta para explicar el rendimiento: la clase Mixto, pese a ser una de las más numerosas, rinde bajo por su heterogeneidad interna, mientras que los cálculos úricos e infecciosos, con un soporte reducido, se distinguen entre sí de forma consistente. La clase COD ilustra el caso más

complejo, en el que se combinan un soporte muy reducido y una proximidad química con Mixto, cuyo detalle se retoma en la sección de limitaciones.

Una de las aportaciones centrales del trabajo es la identificación de la causa que limita el rendimiento. El análisis del espacio de características mediante UMAP, junto con las matrices de confusión y las métricas por clase, apunta de forma consistente a que las clases del núcleo COM–COD–Mixto se solapan considerablemente, y a que este solapamiento, más que la capacidad de los modelos, es el factor que más limita el rendimiento. Dicha conclusión se refuerza por el hecho de que las técnicas de balanceo evaluadas —ponderación, SMOTE y aumentación— no mejoran los resultados: al no actuar sobre la separabilidad de las clases, no atacan la causa real del problema. Ello no implica que tales técnicas carezcan de utilidad en general, sino que no la aportan en este conjunto concreto, condicionado por su tamaño reducido y su alta variabilidad intraclase; la mejora del rendimiento en este dominio debe provenir, por tanto, del enriquecimiento de la representación y no de la modificación de la distribución de muestras.

5.2. Contribuciones del trabajo

El presente trabajo aporta varios elementos diferenciales respecto a los estudios de referencia. En primer lugar, emplea un conjunto de datos clasificado según la taxonomía de Grases *et al.* [3], a diferencia de los trabajos habituales, que utilizan la de Daudon *et al.* [11]; ello supone clases más difíciles, ya que varias agregan subtipos morfológicamente distintos bajo una misma etiqueta, e incorpora dos categorías —mixto e infeccioso— ausentes en los trabajos de referencia.

En segundo lugar, propone un marco que permite comparar de forma directa los resultados obtenidos con imágenes y con parches, mediante la agregación de las predicciones de los parches a nivel de imagen; este análisis mostró que, en este conjunto, el enfoque por parches no ofrece ventaja sobre la clasificación directa de imágenes completas.

En tercer lugar, presenta un modelo híbrido que combina descriptores manuales y representaciones profundas, y que logra una mejora apreciable en el escenario multiclase, siendo el más exigente.

Por último, adopta un protocolo de evaluación estricto —validación cruzada con agrupación por *stone_id*—, que evita el reparto de parches de un mismo cálculo entre entrenamiento y prueba y que, aunque produce métricas más bajas que las de un reparto no agrupado, las hace representativas del rendimiento sobre cálculos no vistos.

5.3. Implicaciones clínicas

Desde una perspectiva clínica, los resultados permiten extraer varias consideraciones. La clase COM, mayoritaria y mejor clasificada, se identifica de forma robusta (*FI* superior al 81 %), un aspecto relevante dado que el oxalato cálcico monohidrato es el componente más frecuente en la práctica clínica; asimismo, los modelos distinguen de forma consistente los cálculos úricos de los infecciosos, sin confusión mutua apreciable, si bien el rendimiento individual de ambas clases todavía es mejorable.

El rendimiento global del sistema es, no obstante, moderado, determina sobre todo por el comportamiento bajo e inestable de COD y por los errores de Mixto hacia COM, de modo que el sistema aún no está en condiciones de servir de apoyo al MCA. Más allá de este límite, conviene abordar de forma explícita una cuestión de fondo: si tiene sentido aplicar la IA a este problema. Los resultados, aunque moderados y condicionados principalmente por las limitaciones del conjunto de datos, indican que sí, pues los modelos evaluados —y en especial las arquitecturas de aprendizaje profundo y el híbrido propuesto— han demostrado capacidad para aprender la separación entre clases y automatizar, al menos parcialmente, la tarea. La vía para mejorar su rendimiento pasa por disponer de un conjunto de datos más amplio, que acercaría estos sistemas a un uso traslacional capaz de conectar los hallazgos de laboratorio con aplicaciones clínicas que mejoren el diagnóstico, el tratamiento y la prevención de la litiasis renal.

5.4. Limitaciones

La limitación principal del trabajo reside en el propio conjunto de datos. Por una parte, su tamaño reducido y el marcado desbalance entre clases, condicionado por la frecuencia natural de aparición de cada tipo de cálculo, dificultan especialmente la clasificación multiclase. Por otra, y de forma más determinante, las clases presentan una alta variabilidad intracase y un solapamiento considerable en el espacio de características que, como se ha mostrado a lo largo de los resultados, es el factor que más limita el rendimiento de los modelos.

En este sentido, conviene distinguir los dos problemas que afectan a la clase COD. Su escaso soporte hace que sus métricas sean muy inestables, un problema de estimación que un conjunto de datos mayor resolvería con seguridad; su confusión con Mixto, en cambio, responde a una proximidad química real entre subtipos oxálicos, un problema de separabilidad que más muestras no resolverían necesariamente si reprodujeran la misma ambigüedad.

Otra limitación afecta al enfoque por parches, ya que el número de parches extraídos no es uniforme entre imágenes —depende del tamaño, del número de fragmentos y de la disposición de cada cálculo—, de modo que no siempre son igualmente representativos del cálculo completo, lo que puede explicar por qué en determinadas imágenes resulta preferible clasificar la imagen completa.

En cuanto al preprocesamiento, la segmentación se ha realizado de manera semiautomática, lo que constituye un cuello de botella: la generación manual de las cajas delimitadoras y las máscaras es un proceso laborioso y de precisión limitada, que dejaría de ser escalable ante una ampliación del conjunto de datos. A esto se le suma la limitada precisión de este enfoque para segmentar correctamente los cálculos formados por varios fragmentos pequeños y solapados.

Desde el punto de vista metodológico, reforzar la fiabilidad de los resultados exigiría repetir los experimentos con varias semillas distintas. Asimismo, las visualizaciones del espacio de características se han generado a partir de los *embeddings* de un único *fold*, con un propósito ilustrativo; una proyección a partir de *embeddings out-of-fold* —empleando todos los *folds* para obtener las predicciones de todas las muestras— sería metodológicamente más rigurosa. Por último, la superioridad de los histogramas de color frente a las energías HSV, aunque consistente en todas las condiciones evaluadas,

se ha establecido a nivel de rendimiento, y determinar el mecanismo exacto de esa diferencia queda como una cuestión abierta.

5.5. Líneas de trabajo futuras

A partir de las limitaciones identificadas y de los resultados obtenidos, se proponen varias líneas de trabajo futuras.

La primera es la automatización de la segmentación: un segmentador automático aceleraría la generación de las máscaras y resultaría de gran utilidad ante la previsible ampliación del conjunto de datos.

La segunda, motivada por la mejora que ha introducido el modelo híbrido, consiste en comparar distintas técnicas de fusión (temprana, intermedia y tardía) sobre este conjunto de datos, así como estudiar mecanismos que equilibren la contribución de las dos ramas, dado que el desequilibrio de dimensionalidad entre el *backbone* (2048 dimensiones) y los descriptores manuales (40 dimensiones) podría estar limitando el aprovechamiento de las características clásicas.

La tercera se deriva del problema central del conjunto de datos: dada la escasa separabilidad de las clases, conviene explorar vías que enriquezcan la representación antes que modificar la distribución de muestras. Una opción prometedora es el uso de LBP multirresolución, que combina patrones de textura a distintas escalas y podría aportar dimensiones discriminativas adicionales sin recurrir a la generación de muestras sintéticas.

La cuarta es la incorporación de técnicas de interpretabilidad para visualizar qué regiones del cálculo resultan determinantes en la decisión.

Finalmente, la ampliación del conjunto de datos constituye la línea más transversal. Un conjunto mayor y con muestras equilibradas para ambas vistas permitiría obtener estimaciones más estables y reduciría la varianza entre *folds*, especialmente en las clases de menor soporte como COD, y haría posible combinar las dos vistas de un mismo cálculo en una única representación (en el caso del XGBoost), siguiendo la línea de los trabajos del estado del arte, para aprovechar mejor la información disponible. Con más datos podría, además, reevaluarse el comportamiento de las técnicas de balanceo en mejores condiciones, si bien —dado que aquí el factor dominante es la separabilidad y no la frecuencia de clases— un conjunto mayor no garantizaría por sí solo que resultaran eficaces.

5.6. Reflexión personal

La realización de este trabajo ha constituido mi primer contacto con la investigación en la clasificación de cálculos renales y, partiendo de una experiencia previa limitada con datos médicos, mi entrada en un dominio clínico concreto que no había abordado hasta entonces. En este recorrido, la lectura de la literatura de referencia ha sido tan valiosa como el propio desarrollo técnico, ya que me ha permitido situar los resultados en relación con las necesidades de la práctica clínica y no como cifras aisladas.

El proyecto me ha mostrado, además, la complejidad de trabajar con datos clínicos reales, donde el desbalanceo, la variabilidad entre muestras y la necesidad de interpretabilidad manifiestan la necesidad de un puente constante entre el rigor de

la inteligencia artificial y su utilidad en el ámbito médico. De este proceso me llevo también una idea que considero central: que entender con precisión qué condiciona el rendimiento de un sistema forma parte esencial del trabajo de investigación, y que reconocer sus limitaciones es parte del mismo aprendizaje.

En conjunto, más allá de reforzar mis conocimientos técnicos, este trabajo me ha situado por primera vez en un contexto de investigación real, con sus incertidumbres y sus limitaciones, del que conservo una base y una motivación sólidas para seguir explorando la aplicación de la inteligencia artificial a los problemas del ámbito biomédico.



ANNEXOS

A.1. Parámetros de los modelos del DL

Tabla A.1: Hiperparámetros utilizados para el entrenamiento del ResNet50 y el modelo híbrido.

Hiperparámetro	Valor
<i>Batch size</i>	8 (Imagen) / 16 (Patch)
<i>Num workers</i>	2
<i>Dropout</i>	0,3
<i>Epochs</i>	30
<i>Learning rate</i>	10^{-4}
<i>Weight decay</i>	10^{-4}
<i>Optimizer</i>	AdamW
<i>Scheduler (Scheduler)</i>	ReduceLROnPlateau
<i>Early stopping</i>	10 épocas

BIBLIOGRAFÍA

- [1] T. Ojala, M. Pietikainen, and T. Maenpaa, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 7, pp. 971–987, 2002. [Online]. Available: <https://doi.org/10.1109/TPAMI.2002.1017623> (document), 2.2.2, 2.1, 2.2.2, 2.2.2
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90> (document), 2.5, 2.2, 2.5, 2.5, 2.3
- [3] F. Grases, A. Costa-Bauzá, M. Ramis, V. Montesinos, and A. Conte, "Simple classification of renal calculi closely related to their micromorphology and etiology," *Clinica Chimica Acta*, vol. 322, pp. 29–36, August 2002. [Online]. Available: [https://doi.org/10.1016/s0009-8981\(02\)00063-3](https://doi.org/10.1016/s0009-8981(02)00063-3) (document), 1, 1.3, 2.1, 2.1.1, 2.1, 3.2, 3.5, 4.5, 5.2
- [4] F. Lopez-Tiro, V. Estrade, J. Hubert, D. Flores-Araiza, M. Gonzalez-Mendoza, G. Ochoa, and C. Daul, "On the in vivo recognition of kidney stones using machine learning," *IEEE Access*, vol. 12, pp. 10 736–10 759, 2024. [Online]. Available: <https://doi.org/10.1109/ACCESS.2024.3351178> (document), 1, 2.2.1, 2.3, 3.2, 3.4.1, 3.6.2, 4, 4.4.2, 4.4.2, 4.5, 4.5, 4.5, 4.11, 4.6
- [5] F. Grases and A. C. Bauzá, "Diagnóstico de la litiasis renal a través del cálculo. estudio morfocomposicional," *Archivos españoles de urología*, vol. 74, no. 1, pp. 35–48, 2021. 1
- [6] I. Sorokin, C. Mamoulakis, K. Miyazawa, A. Rodgers, J. Talati, and Y. Lotan, "Epidemiology of stone disease across the world," *World Journal of Urology*, vol. 35, no. 9, pp. 1301–1320, September 2017. [Online]. Available: <https://doi.org/10.1007/s00345-017-2008-6> 1
- [7] R. A. Vega *et al.*, "Comorbidity and socio-demographic factors associated with renal lithiasis in persons aged 40 to 65: A cross-sectional study," *Medicina Clínica (English Edition)*, vol. 149, no. 9, pp. 383–390, April 2017. [Online]. Available: <https://doi.org/10.1016/j.medcle.2017.09.017> 1
- [8] D. B. Leusmann, H. Niggemann, S. Roth, and H. von Ahlen, "Recurrence rates and severity of urinary calculi," *Scandinavian Journal of Urology and Nephrology*, vol. 29, no. 3, pp. 279–283, September 1995. 1

- [9] H.-G. Tiselius, "Epidemiology and medical management of stone disease," *BJU International*, vol. 91, no. 8, pp. 758–767, May 2003. [Online]. Available: <https://doi.org/10.1046/j.1464-410x.2003.04208.x>. 1
- [10] A. Basiri, M. Taheri, and F. Taheri, "What is the state of the stone analysis techniques in urolithiasis?" *Urology journal*, vol. 9, no. 2, pp. 445–54, 2012. 1
- [11] M. Daudon, C. A. Bader, and P. Jungers, "Urinary calculi: review of classification methods and correlations with etiology," *Scanning Microsc*, vol. 7, no. 3, pp. 1081–1106, September 1993. [Online]. Available: pubmed.ncbi.nlm.nih.gov/8146609/ 1, 4.5, 5.2
- [12] M. Corrales, S. Doizi, Y. Barghouthy, O. Traxer, and M. Daudon, "Classification of stones according to Michel Daudon: A narrative review," *European Urology Focus*, vol. 7, no. 1, pp. 13–21, 2021. [Online]. Available: <https://doi.org/10.1016/j.euf.2020.11.004> 1
- [13] M. Daudon and other, "Comprehensive morpho-constitutional analysis of urinary stones improves etiological diagnosis and therapeutic strategy of nephrolithiasis," *Comptes Rendus Chimie*, vol. 19, no. 11, pp. 1470–1491, 2016. [Online]. Available: <https://doi.org/10.1016/j.crci.2016.05.008> 1
- [14] F. López *et al.*, "Assessing deep learning methods for the identification of kidney stones in endoscopic images," *Proceedings of the IEEE Engineering in Medicine and Biology Society (EMBC)*, pp. 2778–2781, November 2021. [Online]. Available: <https://doi.org/10.1109/EMBC46164.2021.9630211> 1, 1, 2.2.3, 2.3, 4
- [15] J. Serrat and F. Lumbreras and F. Blanco and M. Valiente and M. López-Mesas, "myStone: A System for Automatic Kidney Stone Classification," in *Expert Systems with Applications*, vol. 89, December 2017, pp. 41–51. 1
- [16] K. M. Black, H. Law, A. Aldoukhi, J. Deng, and K. R. Ghani, "Deep learning computer vision algorithm for detecting kidney stone composition," *BJU International*, vol. 125, no. 6, pp. 920–924, February 2020. [Online]. Available: <https://doi.org/10.1111/bju.15035> 1, 3.4.3
- [17] A. Martínez and others, "Towards an automated classification method for ureteroscopic kidney stone images using ensemble learning," in *Proceedings of the 42nd Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, July 2020, pp. 1936–1939. [Online]. Available: <https://doi.org/10.1109/EMBC44109.2020.9176121> 1
- [18] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. [Online]. Available: <https://doi.org/10.1023/A:1010933404324> 1
- [19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 2016, pp. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785> 1, 2.4, 2.4, 2.4, 2.4

-
- [20] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf 1, 1
- [21] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," apr 2015. [Online]. Available: <https://doi.org/10.48550/arXiv.1409.1556> 1, 1
- [22] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," Sep. 2014. [Online]. Available: <https://doi.org/10.48550/arXiv.1409.4842> 1
- [23] V. Estrade, M. Daudon, E. Richard, J.-C. Bernhard, G. Robert, and B. D. de Senneville, "Towards automatic recognition of pure and mixed stones using intra-operative endoscopic digital images," *BJU International*, vol. 129, no. 2, pp. 234–242, February 2022. [Online]. Available: <https://doi.org/10.1111/bju.15515> 1
- [24] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Computer Vision – ECCV 2016*. Springer, 2016, pp. 630–645. [Online]. Available: https://doi.org/10.1007/978-3-319-46493-0_38 1
- [25] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626. [Online]. Available: <https://doi.org/10.1109/ICCV.2017.74> 1
- [26] E. Villalvazo-Avila *et al.*, "Improved kidney stone recognition through attention and multi-view feature fusion strategies," in *2023 IEEE 20th International Symposium on Biomedical Imaging (ISBI)*, 2023, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/ISBI53787.2023.10230794> 1
- [27] F. Lopez-Tiro *et al.*, "Improving automatic endoscopic stone recognition using a multi-view fusion approach enhanced with two-step transfer learning," August 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2304.03193> 1, 2.3
- [28] W. Zhu, R. Zhou, Y. Yao, T. D. Campbell, R. K. Jain, and J. Luo, "Segprompt: Using segmentation map as a better prompt to finetune deep models for kidney stone classification," in *Medical Imaging with Deep Learning*, vol. 227, 2024, pp. 1680–1690. [Online]. Available: <https://proceedings.mlr.press/v227/zhu24b.html> 1
- [29] C. Salazar-Ruiz, F. Lopez-Tiro, I. Reyes-Amezcu, C. Larose, G. Ochoa-Ruiz, and C. Daul, "Evaluation of Few-Shot Learning Methods for Kidney Stone Type Recognition in Ureteroscopy," arXiv preprint arXiv:2505.17921, May 2025. [Online]. Available: <https://arxiv.org/abs/2505.17921> 1
- [30] A. Parnami and M. Lee, "Learning from Few Examples: A Summary of Approaches to Few-Shot Learning," Mar. 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2203.04291> 1

- [31] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818 –2826. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.308> 1
- [32] I. Reyes-Amezcu, F. López-Tiro, G. Ochoa-Ruiz, and C. Daul, “Vit vs cnn: A comparative study for kidney stone classification,” arXiv preprint arXiv:2508.13461, August 2025. [Online]. Available: <https://arxiv.org/abs/2508.13461> 1, 3.4.1
- [33] Dosovitskiy, A. , “An Image Is Worth 16×16 Words: Transformers for Image Recognition at Scale,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, June 2021, pp. 45 –67. 1, 2.6
- [34] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018. 2.2.1, 2.2.1
- [35] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022. [Online]. Available: <https://doi.org/10.1007/978-3-030-34372-9> 2.2.1, 2.2.1
- [36] W. Wong, *Principios del diseño en color*. Editorial Gustavo Gili, 1988. 2.2.1
- [37] T. Ojala, M. Pietikäinen, and D. Harwood, “A comparative study of texture measures with classification based on featured distributions,” *Pattern Recognition*, vol. 29, no. 1, pp. 51–59, January 1996. [Online]. Available: [https://doi.org/10.1016/0031-3203\(95\)00067-4](https://doi.org/10.1016/0031-3203(95)00067-4) 2.2.2
- [38] L. Hou, D. Samaras, T. Kurc, Y. Gao, J. Davis, and J. Saltz, “Patch-based convolutional neural network for whole slide tissue image classification,” ser. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. IEEE Computer Society, dec 2016, pp. 2424 –2433. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.266> 2.3
- [39] L. Shen *et al.*, “Deep learning to improve breast cancer detection on screening mammography,” *Scientific Reports*, vol. 9, p. 12495, August 2019. 2.3
- [40] G. Litjens *et al.*, “A survey on deep learning in medical image analysis,” *Medical Image Analysis*, vol. 42, pp. 60–88, December 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1361841517301135> 2.3
- [41] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009. 2.4, 3.9
- [42] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning with Applications in Python*. Springer, 2021. 2.4
- [43] A. Natekin and A. Knoll, “Gradient boosting machines, a tutorial,” *Frontiers in Neurorobotics*, vol. 7, 2013. [Online]. Available: <https://www.frontiersin.org/journals/neurorobotics/articles/10.3389/fnbot.2013.00021> 2.4, 2.4
- [44] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189 –1232, 2001. 2.4

- [45] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. 2.5, 2.5, 2.5
- [46] R. Yamashita, M. Nishio, R. K. G. Do, and K. Togashi, "Convolutional neural networks: an overview and application in radiology," *Insights Imaging*, vol. 9, pp. 611–629, August 2018. [Online]. Available: <https://doi.org/10.1007/s13244-018-0639-9> 2.5, 2.5, 2.5, 2.5
- [47] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, 2010, pp. 807–814. 2.5
- [48] Ioffe, Sergey and Szegedy, Christian, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, vol. 37, July 2015. 2.5, 2.5
- [49] D. Ramachandram, Taylor, and G. W., "Deep multimodal learning: A survey on recent advances and trends," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 96–108, November 2017. 2.5
- [50] J. Ma, Y. He, F. Li, L. Han, C. You, and B. Wang, "Segment anything in medical images," *Nature Communications*, vol. 15, no. 1, January 2024. [Online]. Available: <https://doi.org/10.1038/s41467-024-44824-z> 2.6, 2.6, 3.3
- [51] Y. Bengio, A. Courville, and P. Vincent, "Representation Learning: A Review and New Perspectives," Apr. 2014. [Online]. Available: <https://doi.org/10.48550/arXiv.1206.5538> 2.6
- [52] U. Michelucci, "An Introduction to Autoencoders," Jan. 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2201.03898> 2.6, 2.6
- [53] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," aug 2017. [Online]. Available: <https://doi.org/10.48550/arXiv.1706.03762> 2.6, 2.6, 2.6
- [54] Kirillov, Alexander and Mintun, Eric and Ravi, Nikhila and Mao, Hanzi and Rolland, Chloe and Gustafson, Laura and Xiao, Tete and Whitehead, Spencer and Berg, Alexander C. and Lo, Wan-Yen and Dollár, Piotr and Girshick, Ross, "Segment Anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Paris, France, October 2023. 2.6
- [55] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, jun 2002. [Online]. Available: <https://doi.org/10.1613/jair.953> 3.7.1
- [56] M. Z. Alam, T. Roy, Kawsar, H. M. Nahid, and I. Rimi, "Enhancing transfer learning for medical image classification with smote: A comparative study," *arXiv.org*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.20235> 3.7.1

- [57] A. Halim, M. Dwifabri, and F. Nhita, "Handling imbalanced data sets using smote and adasyn to improve classification performance of ecoli data sets," *Building of Informatics, Technology and Science (BITS)*, vol. 5, no. 1, January 2023. [Online]. Available: <https://doi.org/10.47065/bits.v5i1.3647> 3.7.1
- [58] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing 'I&' Management*, no. 4, pp. 427–437, July 2009. [Online]. Available: <https://doi.org/10.1016/j.ipm.2009.03.002> 3.8
- [59] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009. [Online]. Available: <https://doi.org/10.1109/TKDE.2008.239> 3.8
- [60] T. Fawcett, "An introduction to roc analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, June 2006. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016786550500303X> 3.8
- [61] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Association for Computing Machinery, 2019, p. 2623–2631. [Online]. Available: <https://doi.org/10.1145/3292500.3330701> 3.10
- [62] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," arXiv:1802.03426, September 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.1802.03426> 3.11